VisionGuard: AI vs Real Image Classification and MLOps Infrastructure

Student: Shayan Ahmed

Roll Number: 23L-0801

GitHub Repository: https://github.com/shayanO-O/23L-0801_A2_Ops

DagsHub Project: https://dagshub.com/shayanO-O/23L-0801_A2_Ops

Live Cloud API URL: http://3.24.214.144

Graded Final Commit Hash: 85d9ace

1. Task 1: Data Understanding and Versioning

1.1 Environment

Exact package versions are pinned in requirements-train.txt, requirements-prod.txt and requirements-dev.txt. This keeps results the same on local Windows, Google Colab and production Linux containers. The main stack is Python 3.11, TensorFlow 2.21.0, Keras 3.13.2 and MLflow 3.18.0. No GPU was detected on the local machine, so TensorFlow runs in CPU mode. The check script confirms that all installed versions match the pins (Figure 1).

Package	Version	Used for
Python	3.11.9	Local, Docker, CI and cloud
TensorFlow	2.21.0	Training and serving (CPU)
Keras	3.13.2	High-level model API
MLflow	3.18.0	Local and remote tracking
DVC (with s3)	3.59.0	Data versioning
Flask	3.1.0	Web application
Scikit-Learn	1.6.1	Evaluation metrics

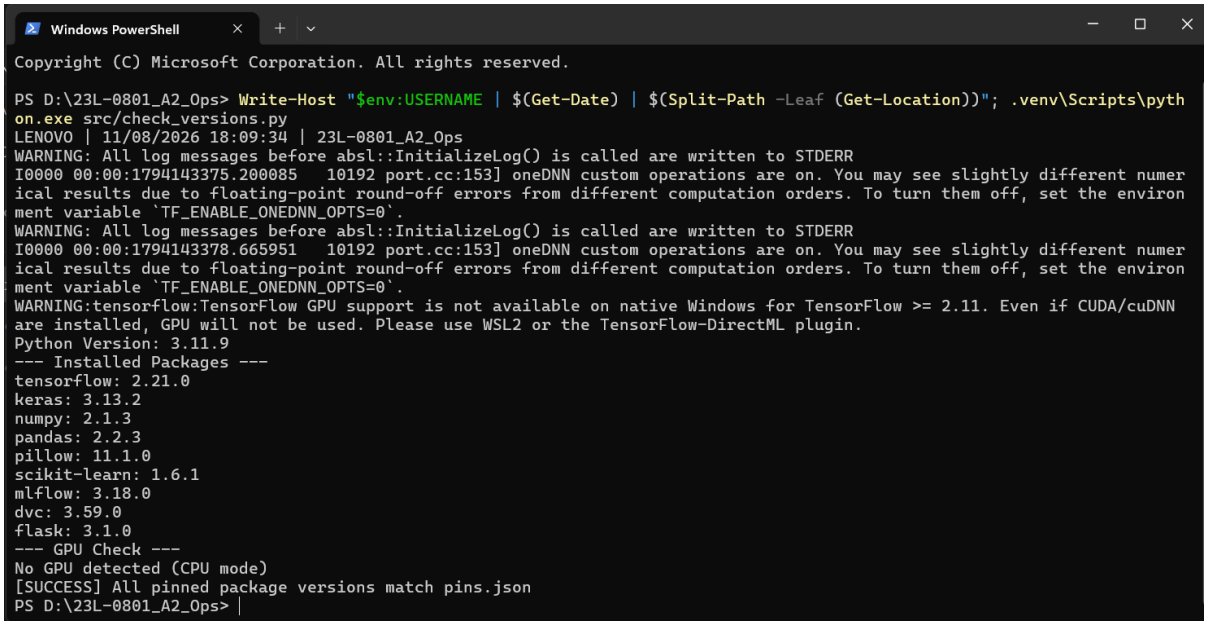


Figure 1: Environment versions and GPU check

1.2 Dataset Audit

The raw dataset has 975 files in data/raw/ across two classes: ai (label 1) and real (label 0). The audit script (src/audit.py) checks that each image can be decoded, looks for zero-byte files and finds exact duplicates using SHA-256 hashes. It also checks for the same image appearing in both classes.

Metric	Value	Action taken
Total scanned files	975	All audited
Corrupted or unreadable files	0	None dropped
Exact duplicates	2 (within one class)	First copy kept, extra copy flagged
Cross-class label collisions	0	None found
Final usable files	974 (538 AI, 436 Real)	Master index created

The table below gives the minimum, median and maximum of image width, height and file size for each class. Real images vary far more in size, up to 8,495 x 7,091 px. The median AI file (195.1 KB) is about 38% larger than the median real file (141.5 KB).

Class	Images	Width in px (min / median / max)	Height in px (min / median / max)	File size (min / median / max)
AI	539	225 / 1,024 / 6,000	135 / 800 / 4,001	11.5 KB / 195.1 KB / 5.37 MB
Real	436	183 / 920 / 8,495	158 / 720 / 7,091	11.0 KB / 141.5 KB / 9.72 MB

1.3 Visual EDA

The EDA script (src/eda.py) produced sample grids and histograms of file size, width and height for each class (Figure 2 to Figure 6). The sample grids show that real images come in many shapes and sizes. The histograms show that real images vary much more in width and height (from 183 px to 8,495 px wide), while AI images are more concentrated around a median of 1024x800. AI images also tend to have larger files, likely because of the fine texture detail that diffusion models produce.



Figure 2: Sample AI-generated images

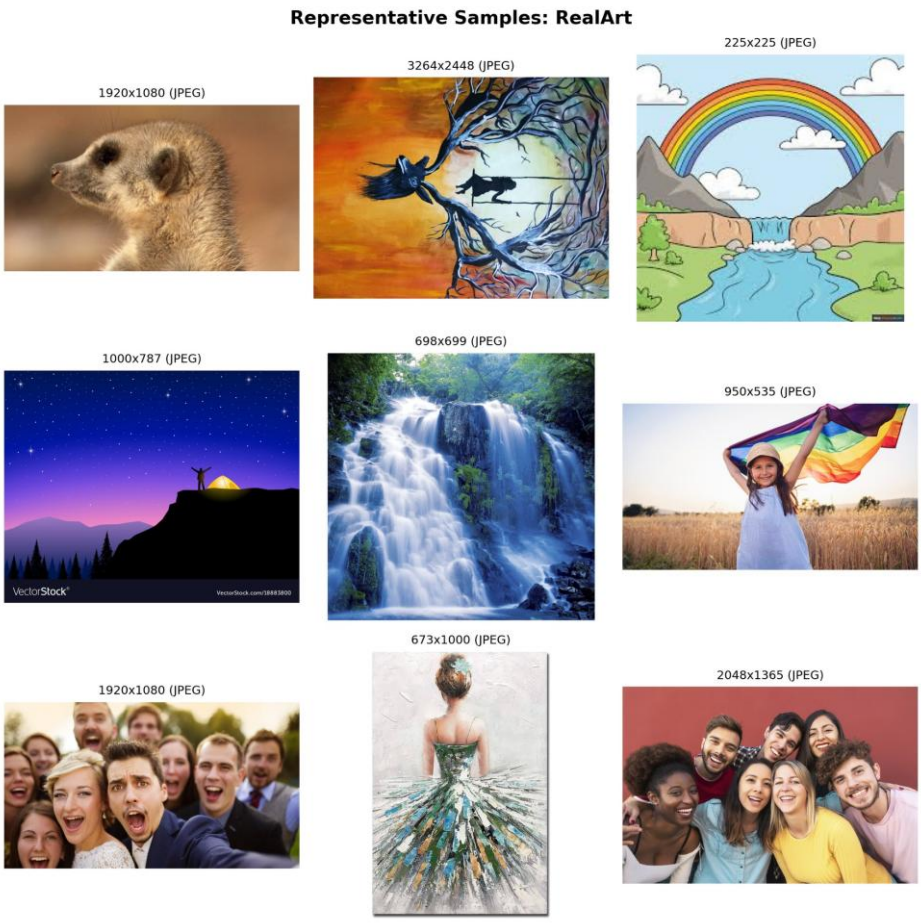


Figure 3: Sample real images

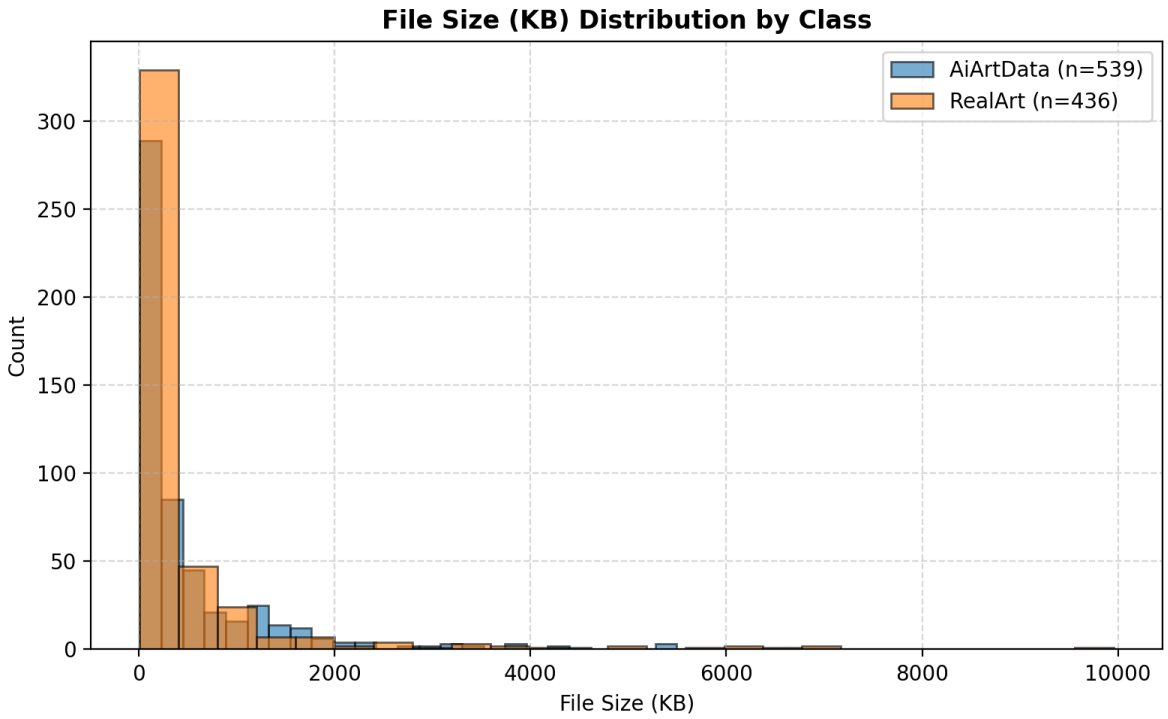


Figure 4: File size distribution by class

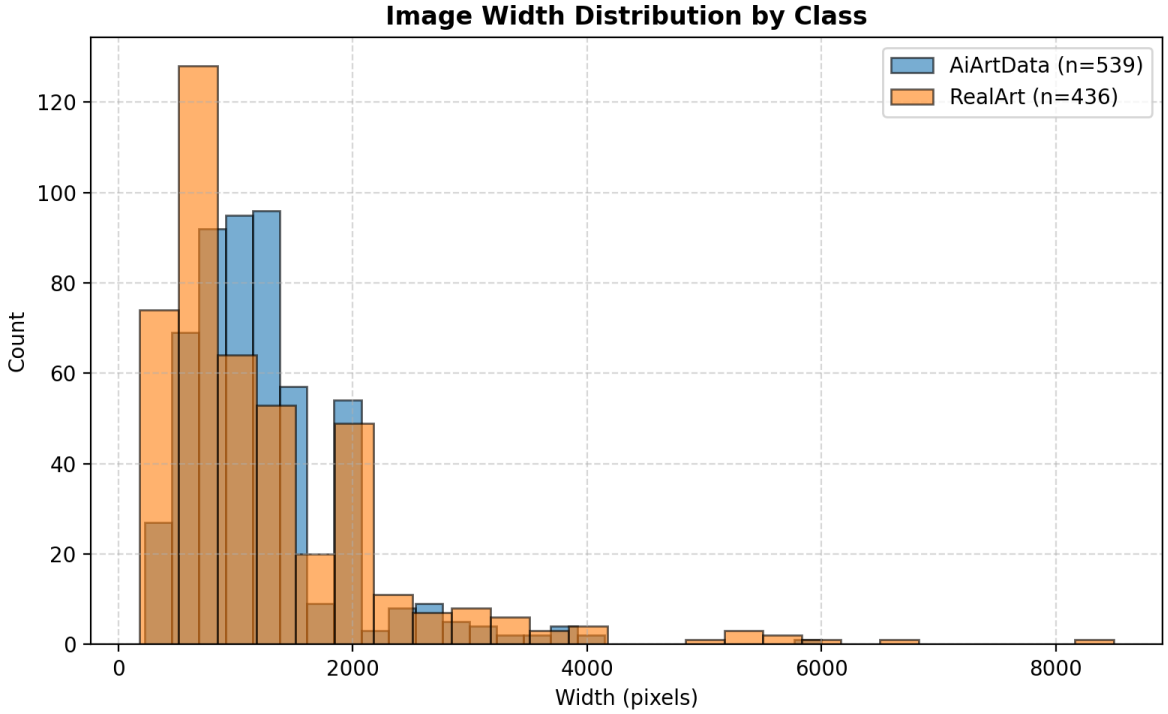


Figure 5: Width distribution by class

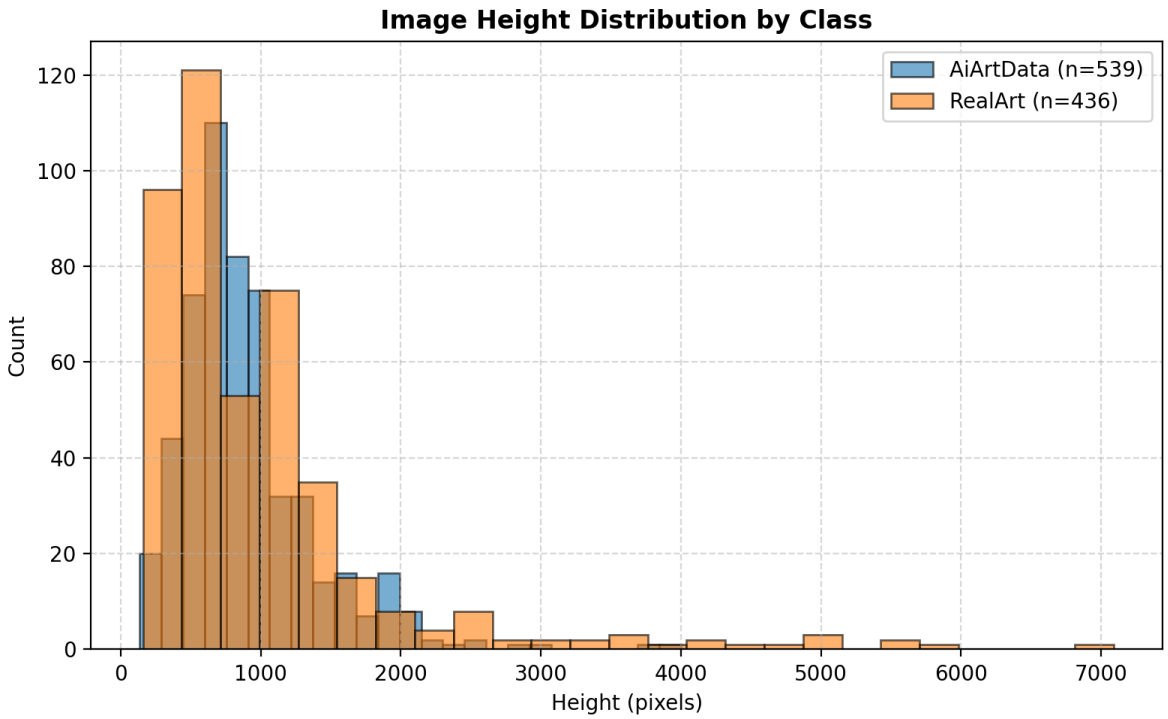


Figure 6: Height distribution by class

Image shape and file size differ between the classes, so a model could use them as shortcuts instead of learning image content. To reduce this risk, all images are resized to 224x224 with the same bilinear step. The data was then split 70% train (681 images), 15% validation (146 images) and 15% test (147 images) with SEED=42. The class ratio is kept in all three splits (splits/train.csv, splits/val.csv and splits/test.csv).

1.4 DVC (Data Version Control)

Git is not suited to large binary files because they bloat the repository, slow down clones and cannot be merged. DVC solves this by keeping the data in separate storage and putting only a small pointer file in Git. Here, Git tracks data/raw.dvc (112 bytes). It records the directory MD5 (8f23d6dc9b06d363c3981642c50856c0.dir), the total size (503,783,378 bytes) and the file count (975). The data itself is stored in DagsHub storage.

A toy workflow was done first in a separate folder. Two versions of a small CSV were committed and tracked with DVC. The older version was restored with git checkout followed by dvc checkout. The data was then pushed to DagsHub, deleted locally and restored with dvc pull (Figure 7 to Figure 9).

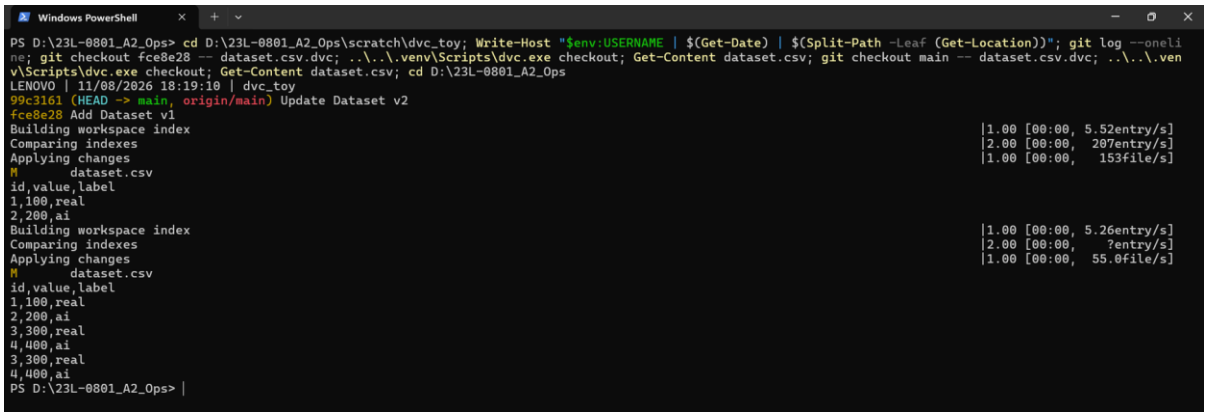


Figure 7: DVC toy commits and rollback to the older version

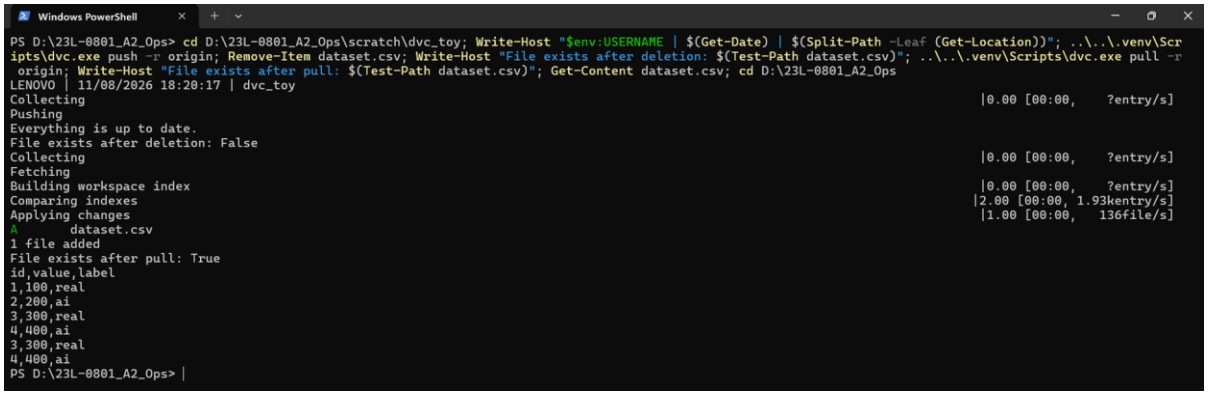


Figure 8: DVC toy push, local delete and pull

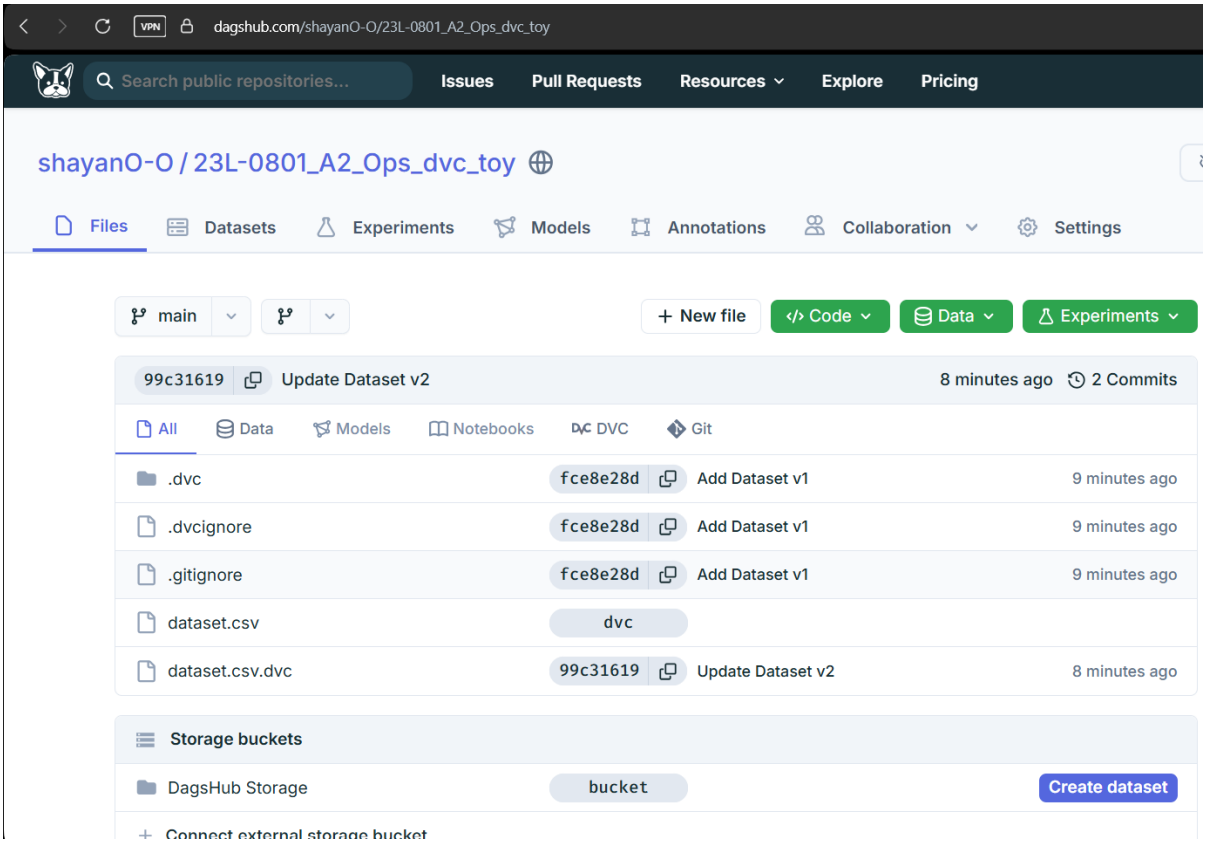


Figure 9: DagsHub toy repository with the tracked CSV

The same steps were then applied to the real dataset in data/raw, which was committed to DVC and pushed to DagsHub (Figure 10 and Figure 11).



Figure 10: DVC pointer details and sync status for the main dataset

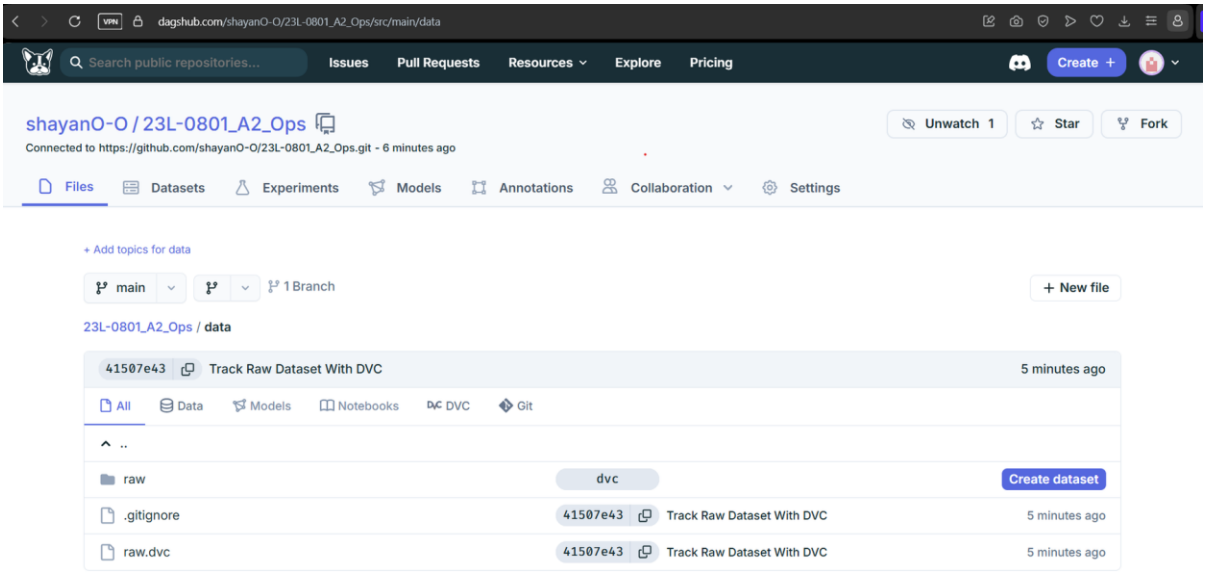


Figure 11: DagsHub repository showing the tracked dataset

2. Task 2: Model Development with TensorFlow/Keras

2.1 Input Pipeline

The pipeline in src/data.py uses tf.data.Dataset and reads the split lists. The helper decode_and_resize decodes each image and resizes it to 224x224 with bilinear interpolation. Images are then batched and prefetched with tf.data.AUTOTUNE. The same helper is used in training, local Flask serving and cloud inference, so training and serving preprocessing stay identical. Random augmentation is applied during training only. It uses a horizontal flip, a rotation of up to 10% and a zoom of up to 10%. Figure 12 shows one augmented batch.

The size 224x224 is the standard input size of MobileNetV2. Resizing stretches non-square images into a square, so the aspect ratio changes and circles can become ellipses. Padding was rejected because black bars could become a shortcut if the classes have different aspect ratios. Center cropping was rejected because it throws away edge content such as signatures and watermarks.

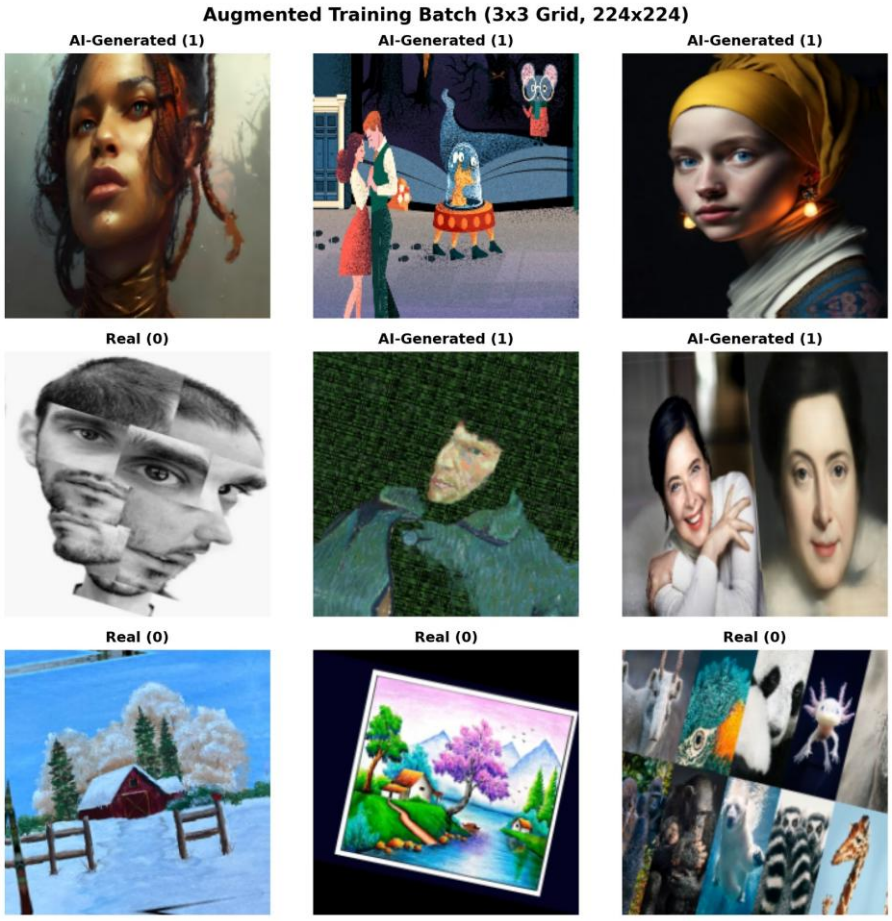
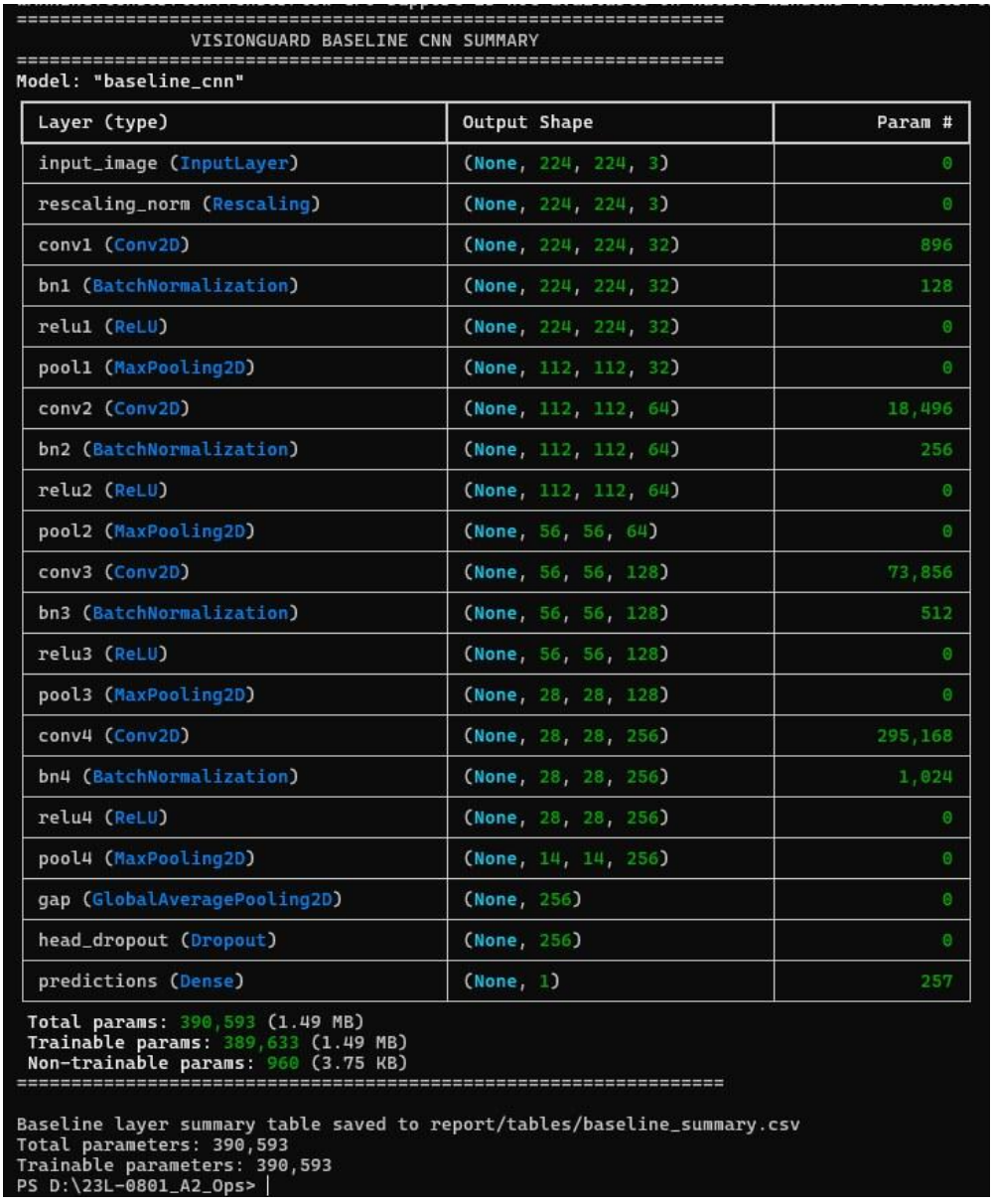


Figure 12: One augmented training batch

2.2 Baseline CNN

The baseline CNN (src/models.py) has four convolution blocks. Each block has Conv2D, BatchNorm, ReLU and MaxPool2D (2x2). The filters are 32, 64, 128 and 128, all with 3x3 kernels. After the blocks come GlobalAveragePooling2D, Dropout (0.4) and one sigmoid output unit. The model has 390,593 parameters. Figure 13 shows the layer summary with output shapes and parameter counts.

The loss is binary cross-entropy because the model has one sigmoid output that gives the probability of AI. The optimizer is Adam with a learning rate of 1e-3. It adapts the step size for each weight and works well without much tuning. Accuracy, precision, recall and ROC-AUC are tracked. ROC-AUC judges how well the model ranks AI images above real ones at every threshold. Precision shows how many AI flags are correct and recall shows how many AI images are caught.



VISIONGUARD BASELINE CNN SUMMARY

Model: "baseline_cnn"

Layer (type)	Output Shape	Param #
input_image (InputLayer)	(None, 224, 224, 3)	0
rescaling_norm (Rescaling)	(None, 224, 224, 3)	0
conv1 (Conv2D)	(None, 224, 224, 32)	896
bn1 (BatchNormalization)	(None, 224, 224, 32)	128
relu1 (ReLU)	(None, 224, 224, 32)	0
pool1 (MaxPooling2D)	(None, 112, 112, 32)	0
conv2 (Conv2D)	(None, 112, 112, 64)	18,496
bn2 (BatchNormalization)	(None, 112, 112, 64)	256
relu2 (ReLU)	(None, 112, 112, 64)	0
pool2 (MaxPooling2D)	(None, 56, 56, 64)	0
conv3 (Conv2D)	(None, 56, 56, 128)	73,856
bn3 (BatchNormalization)	(None, 56, 56, 128)	512
relu3 (ReLU)	(None, 56, 56, 128)	0
pool3 (MaxPooling2D)	(None, 28, 28, 128)	0
conv4 (Conv2D)	(None, 28, 28, 256)	295,168
bn4 (BatchNormalization)	(None, 28, 28, 256)	1,024
relu4 (ReLU)	(None, 28, 28, 256)	0
pool4 (MaxPooling2D)	(None, 14, 14, 256)	0
gap (GlobalAveragePooling2D)	(None, 256)	0
head_dropout (Dropout)	(None, 256)	0
predictions (Dense)	(None, 1)	257

Total params: 390,593 (1.49 MB)
Trainable params: 389,633 (1.49 MB)
Non-trainable params: 960 (3.75 KB)

Baseline layer summary table saved to report/tables/baseline_summary.csv
Total parameters: 390,593
Trainable parameters: 390,593
PS D:\23L-0801_A2_Ops>

Figure 13: Baseline CNN model summary

2.3 Transfer Learning

The dataset is small (974 images), so MobileNetV2 pretrained on ImageNet was used. In the first stage the base is frozen and only the new head is trained. The head is GlobalAveragePooling2D, Dropout (0.4) and a sigmoid Dense layer. Inputs go through mobilenet_v2.preprocess_input, which scales pixels to [-1, 1] as the pretrained weights expect. In the second stage the last block (from block_16_expand onwards) is unfrozen and trained with a smaller learning rate of 1e-4. The table lists the trainable layers in each stage. Stage 1 trains 1,281 of 2,259,265 parameters. Stage 2 trains 880,321 parameters. The BatchNorm layers inside the unfrozen block stay frozen, so small batches do not disturb their stored statistics.

Stage	Part	Layers	Trainable layers	Trainable params	Non-trainable params
1	MobileNetV2 base (Conv1 to Conv_1)	154	0	0	2,257,984
1	Head (GAP, Dropout, Dense)	3	3	1,281	0
2	Base, frozen part (Conv1 to block_15_project)	140	0	0	1,376,128
2	Base, unfrozen part (block_16_expand to Conv_1)	14	10	879,040	2,816
2	Head (GAP, Dropout, Dense)	3	3	1,281	0

Figure 14 and Figure 15 show the training curves of each stage. Stage 1 (run R1, learning rate 1e-3) has its best epoch at 7. Stage 2 (run R5, learning rate 1e-4) has its best epoch at 8. The best epoch is marked in each plot.

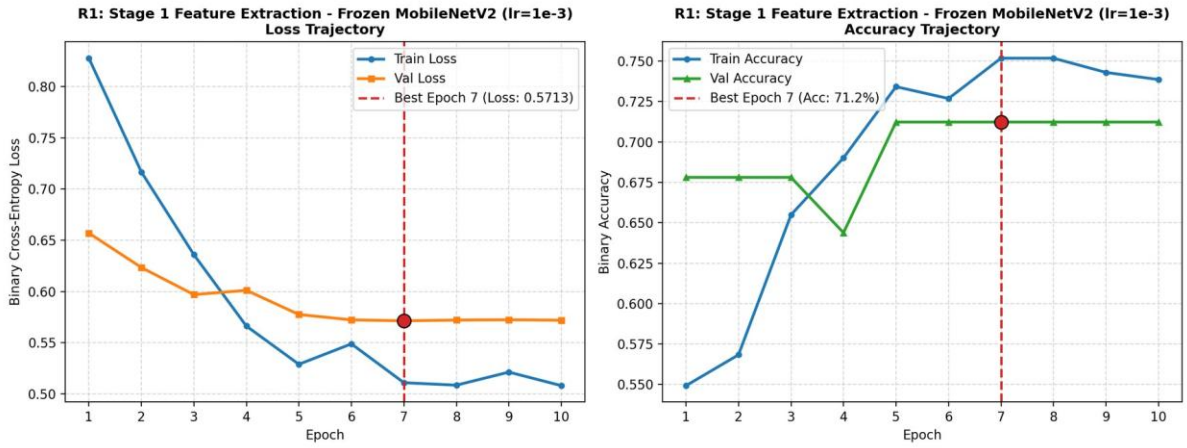


Figure 14: Stage 1 training curves (run R1, learning rate 1e-3)

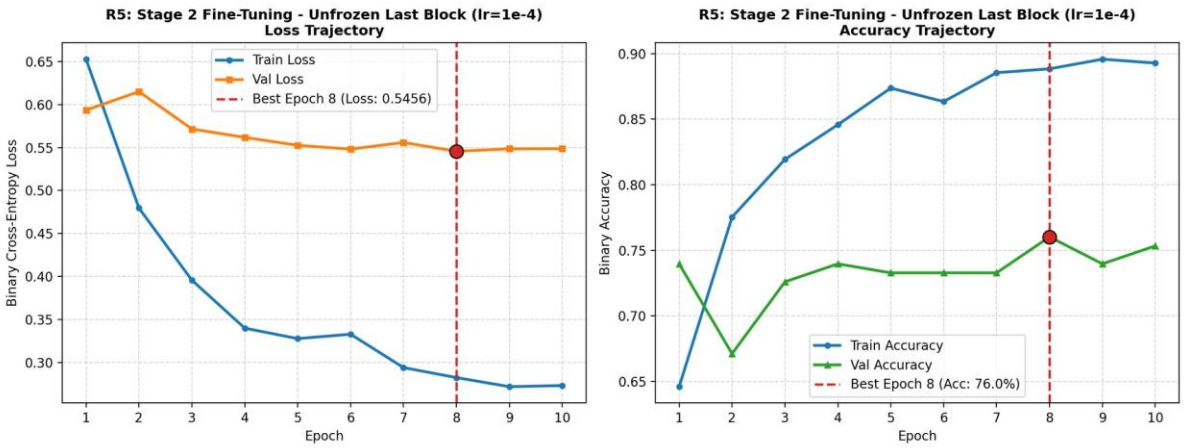


Figure 15: Stage 2 training curves (run R5, learning rate 1e-4)

2.4 Training Discipline

The seed is fixed to 42 in Python, NumPy and TensorFlow. All runs use these callbacks:

- EarlyStopping on val_loss with patience 5 and restore_best_weights=True.
- ReduceLROnPlateau on val_loss with factor 0.5, patience 3 and min_lr 1e-6.
- ModelCheckpoint saving the best weights to model/champion.keras.

The best epochs are marked in Figure 14 and Figure 15. The champion run (final_manual) has its best epoch at 6, with a validation loss of 0.5376. The best epoch of every run is listed in section 3.5.

2.5 Evaluation

The test set was used once, after training was finished (src/evaluate.py). For the champion model the decision threshold was chosen on the validation split only. The default is 0.50. The chosen value is 0.5252, which maximizes Youden's J (sensitivity + specificity - 1) on the validation ROC curve. The test results are in the table below. The ROC curves and confusion matrices are in Figure 16 and Figure 17.

Model	Threshold	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Baseline CNN	0.4619	57.14%	65.52%	46.91%	0.5468	0.5683	0.5926
MobileNetV2	0.5000	76.19%	78.05%	79.01%	0.7853	0.8442	0.8618
MobileNetV2 (tuned threshold)	0.5252	75.51%	77.78%	77.78%	0.7778	0.8442	0.8618

The baseline CNN uses its own Youden-optimal threshold of 0.4619. Its scores lie between 0.425 and 0.483, so a threshold of 0.50 would label every image real. The baseline is much weaker than the champion, with a ROC-AUC of 0.5683 against 0.8442.

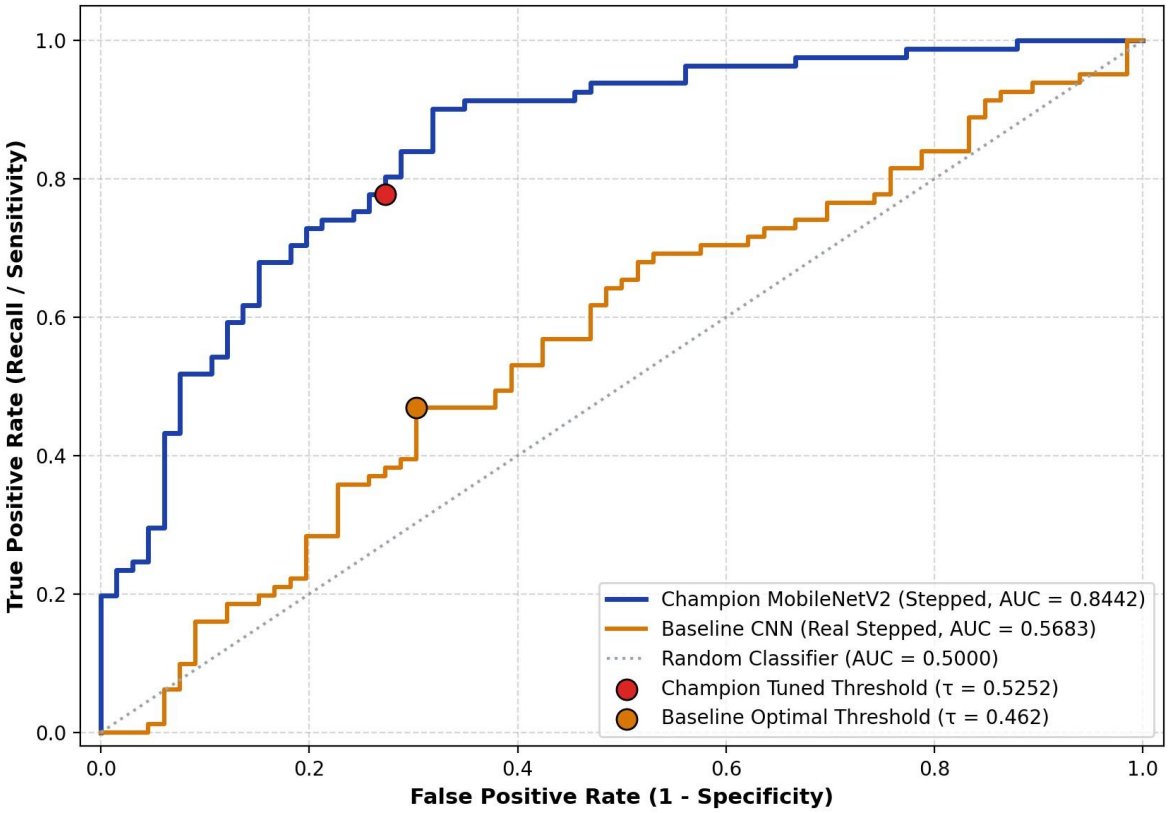


Figure 16: Test set ROC curves (147 test images)

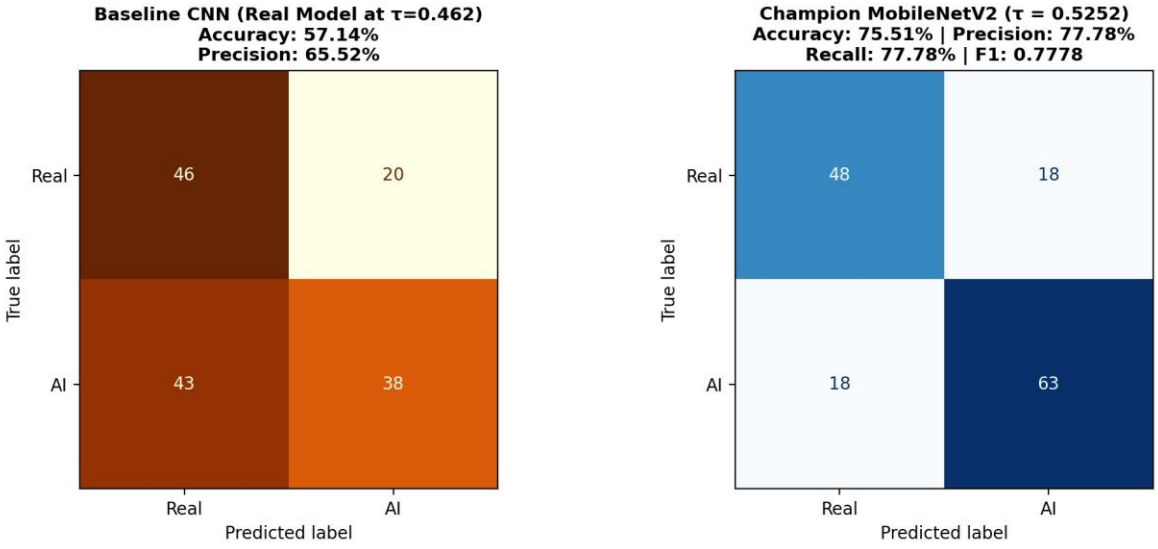


Figure 17: Test set confusion matrices for the baseline CNN and the champion

Calling a real image fake is the worse error. It falsely accuses the creator of an original work and damages trust in the product. Missing an AI image is less harmful. The threshold of 0.5252 is slightly stricter than 0.50 and gives 18 errors of each type on the test set (Figure 17). If false accusations matter even more, the threshold can be raised further.

2.6 Error Analysis

The script src/errors.py collected the misclassified test images. Figure 18 shows 10 of them. Real images wrongly called AI were mostly high-contrast digital art, stylized illustrations and portraits with smooth blurred backgrounds. The model likely mistook this smoothness for AI output. AI images wrongly called real had subdued lighting, photo-like grain and natural skin texture. These images likely lacked the fine distortions the model relies on.

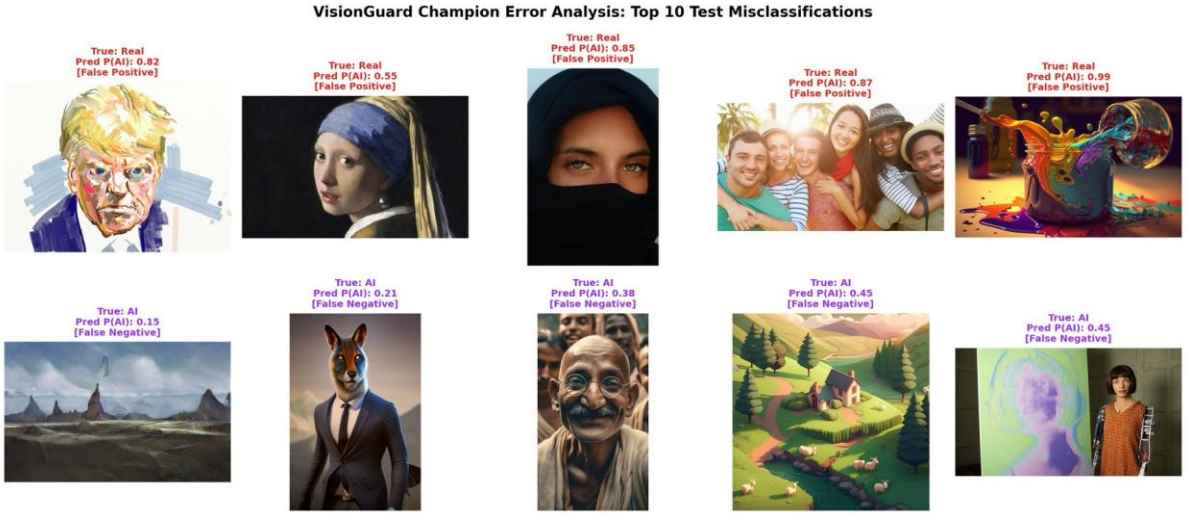


Figure 18: Ten misclassified test images

2.7 Hyperparameter Tuning (Keras Tuner)

A toy search on Fashion-MNIST was run first to test the tuner. It had 4 trials over the number of units (64, 96 and 128) and the learning rate (0.01 and 0.001). The best trial used 96 units and a learning rate of 0.01. The accuracy range was only 1.5 points, so no single setting clearly dominated.

Rank	Units	Learning rate	Val accuracy	Val loss
1	96	0.01	71.5%	0.7812
2	128	0.001	71.5%	0.7954
3	64	0.01	70.0%	0.8241
4	128	0.01	70.0%	0.8409

An 8-trial search was run with src/tuner.py. It tried learning rates of 1e-2, 1e-3 and 1e-4, dropout rates of 0.2, 0.4 and 0.5 and two pooling options (GlobalAveragePooling2D or Flatten plus Dense). Trial 4 gave the best validation AUC of 0.8420. It used a learning rate of 1e-3, dropout 0.4 and GlobalAveragePooling2D. This search did not vary the number of unfrozen layers.

Choosing the best validation score can overfit the validation set. With many trials, the winning setting may fit the quirks of the small validation set (146 images) by chance, so its score is too optimistic. This is why the test set was kept for one final check.

3. Task 3: Experiment Tracking with MLflow and DagsHub

3.1 MLflow Locally

A toy experiment (src/toy_mlflow.py) was run twice with learning rates of 0.01 and 0.001. Parameters, per-epoch metrics and the model were logged, and the two runs were compared in the local MLflow interface (Figure 19 to Figure 21). The terms used by MLflow are explained below.

- **Experiment:** a named group of related runs.
- **Run:** one execution of the training code with everything it logged.
- **Parameter:** a setting fixed before training, such as the learning rate.
- **Metric:** a number measured during or after training, such as validation accuracy.
- **Artifact:** a file saved with a run, such as a plot or the model.
- **Tag:** a free-text label used to find and filter runs.

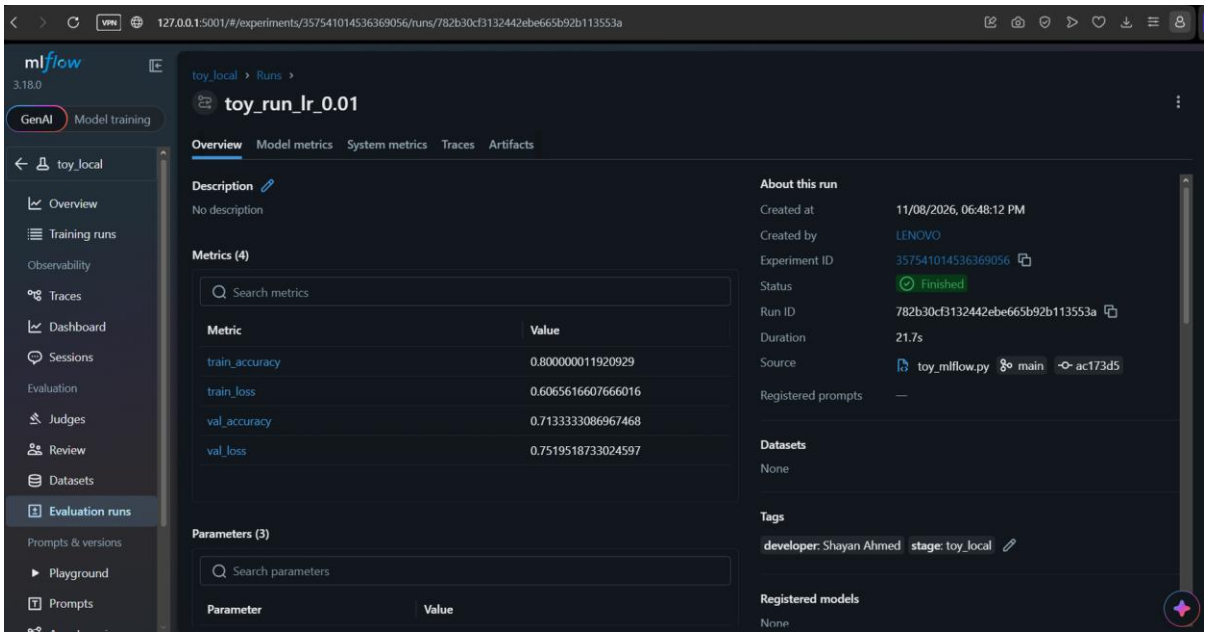


Figure 19: Local run 1 (learning rate 0.01)

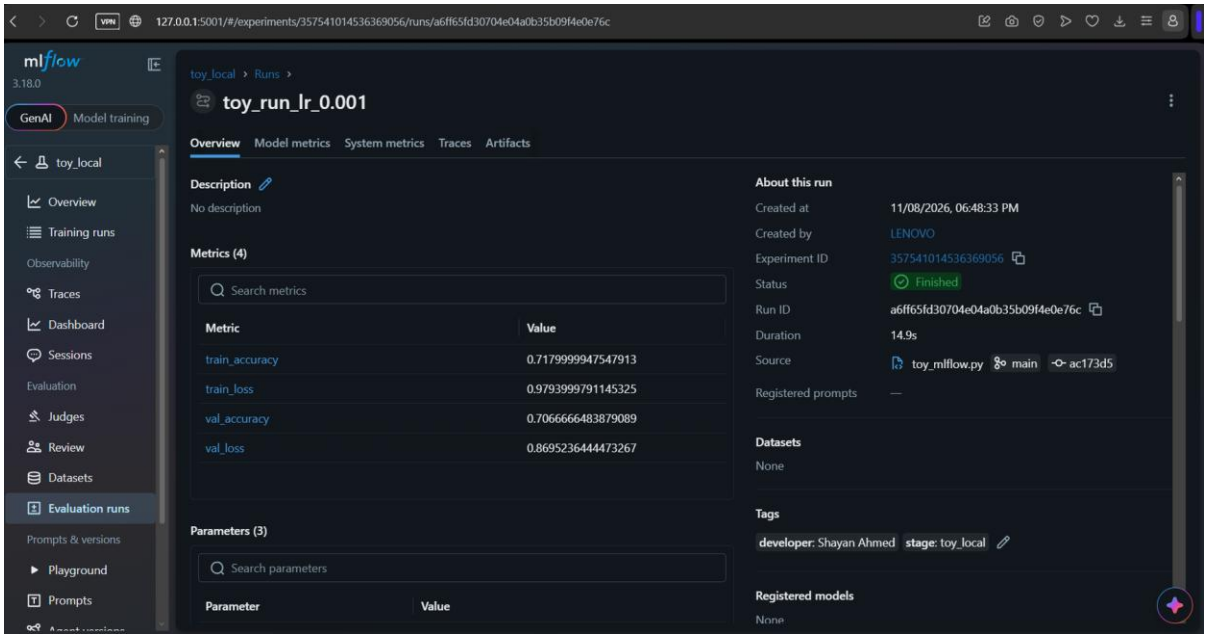


Figure 20: Local run 2 (learning rate 0.001)

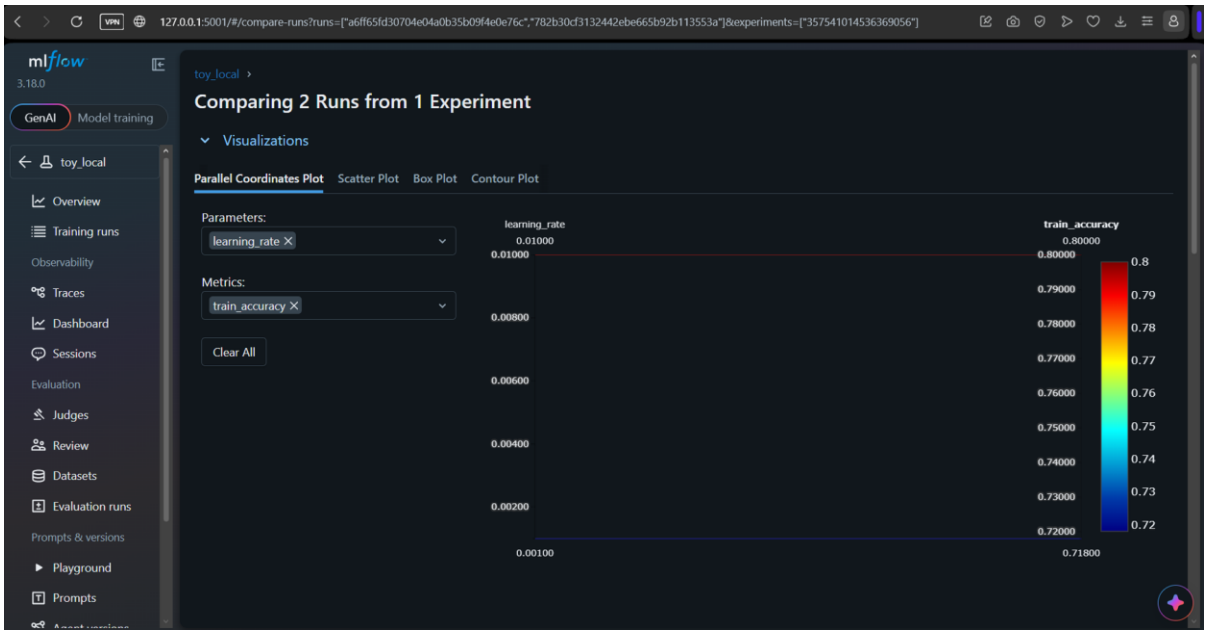


Figure 21: Local comparison of the two runs

3.2 DagsHub Setup

Remote tracking is hosted on DagsHub at https://dagshub.com/shayanO-O/23L-0801_A2_Ops.mlflow. The tracking address and access token are read from environment variables in a .env file. Figure 24 shows that .env is ignored by Git. The toy run was repeated against DagsHub (Figure 22 and Figure 23).

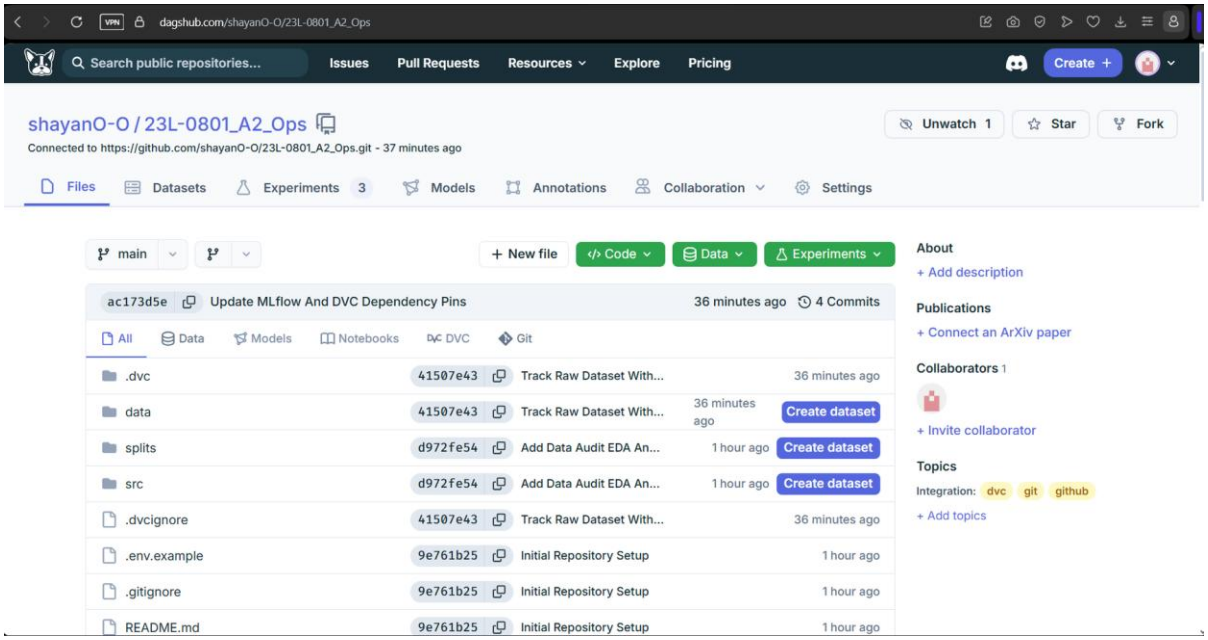


Figure 22: DagsHub project overview

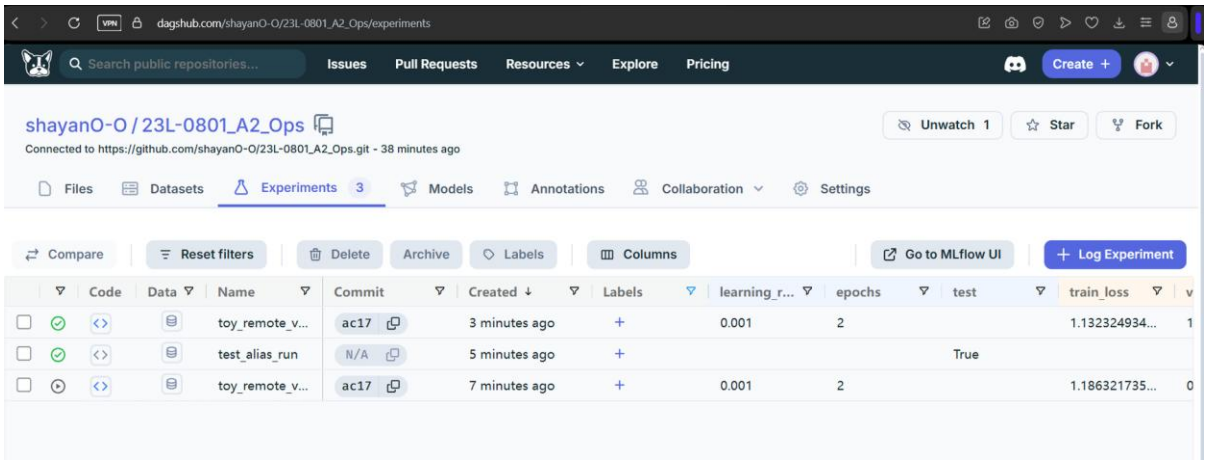


Figure 23: Toy run on DagsHub

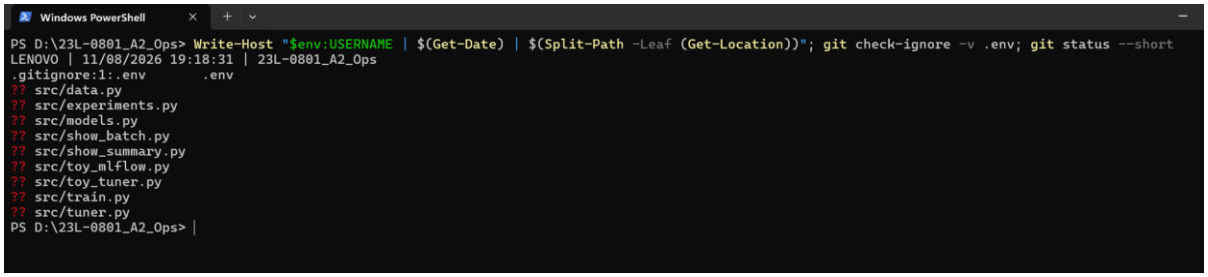


Figure 24: Git ignore check for the .env file

3.3 Instrumenting VisionGuard Training

Training was logged in two ways. The runs are shown in Figure 25 and Figure 26.

- **Manual logging (final_manual):** a Keras callback logged chosen metrics (val_auc, val_precision and val_recall), the hyperparameters, custom tags and the model. It records exactly what was chosen but needs more code.
- **Autologging (final_autolog):** MLflow captured parameters, architecture details and metrics automatically. It needs no extra code but produced many noisy metric keys.

Every run also carries reproducibility tags. The champion run has these values:

Tag	Value
dvc_data_version	8f23d6dc9b06d363c3981642c50856c0.dir
seed	42

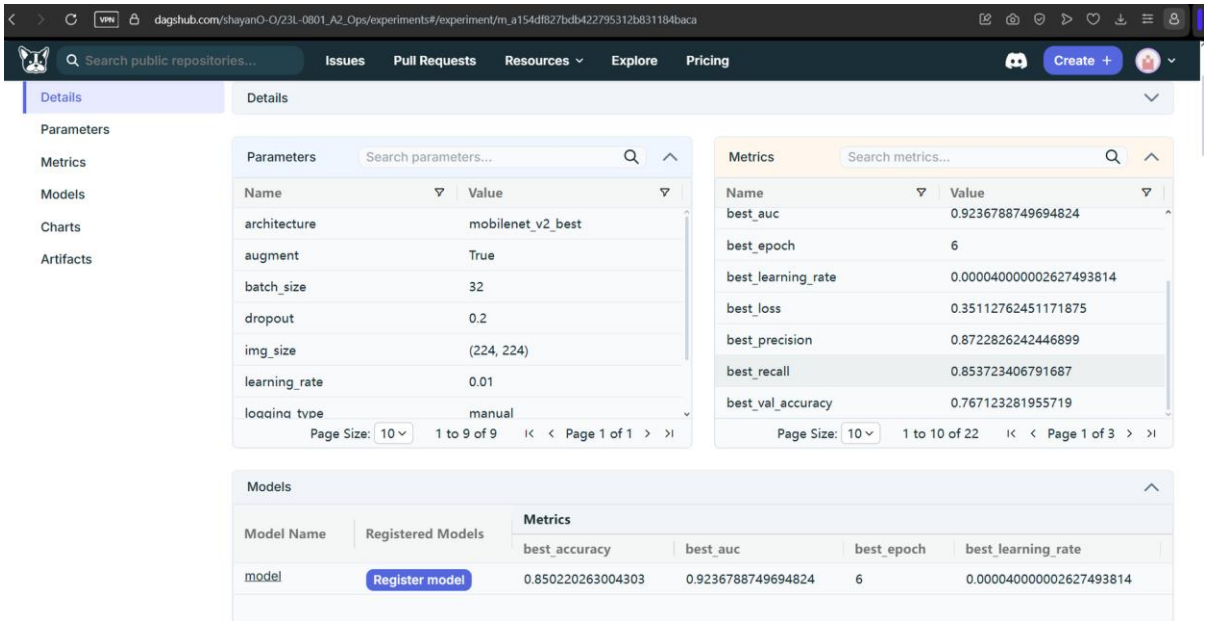


Figure 25: Manual logging run on DagsHub

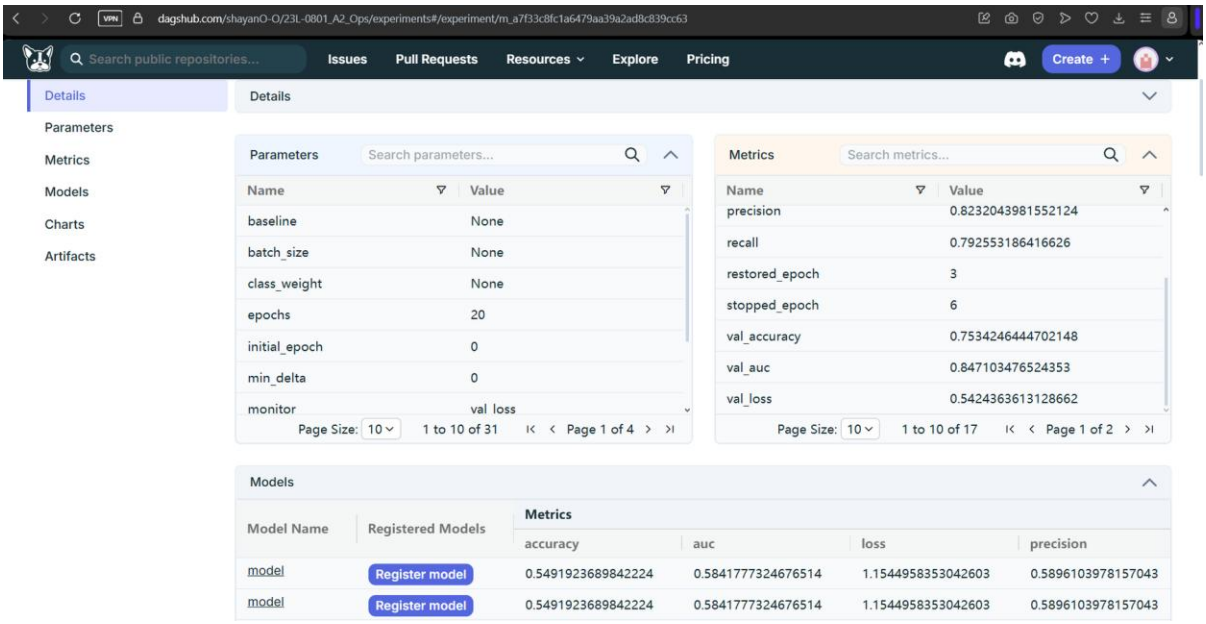


Figure 26: Autologging run on DagsHub

3.4 Artifacts

The champion run stores the ROC curve, confusion matrix, split lists, pip requirements and the saved Keras model (Figure 27).

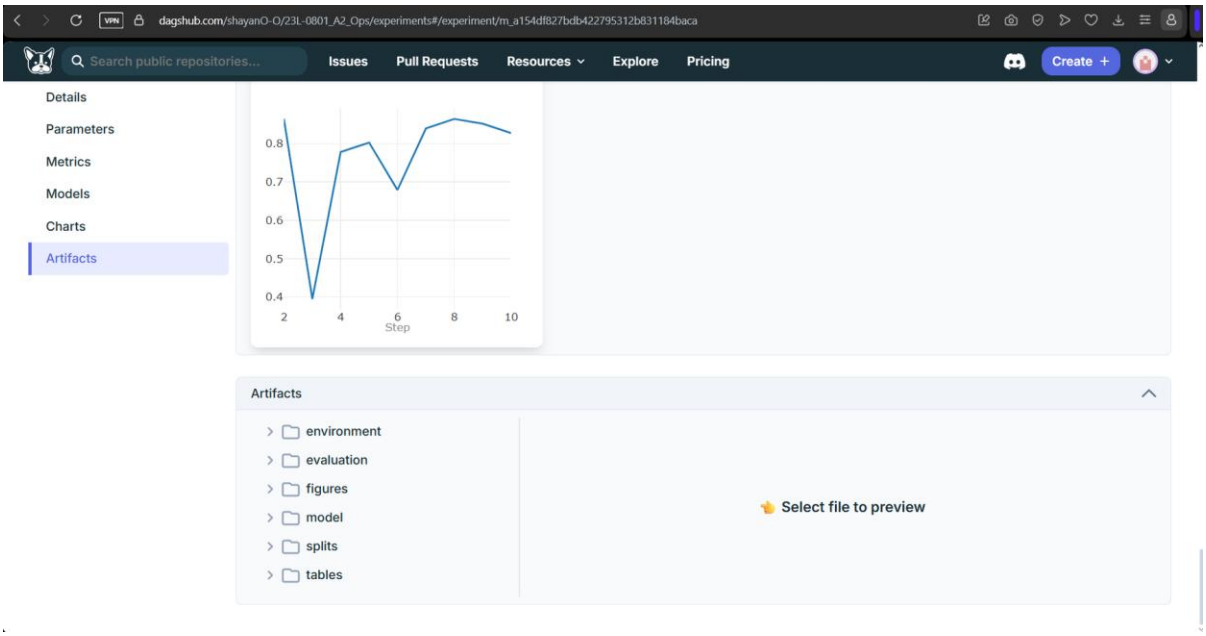


Figure 27: Artifacts of the champion run on DagsHub

3.5 Experiment Comparison

Nine runs were tracked on DagsHub. Each differs from another by one change such as learning rate, augmentation, image size, frozen layers or backbone. The results are in the table and in Figure 28.

Run	Architecture	LR	Best epoch	Val loss	Val acc	Val AUC	Val F1
R0_baseline_cnn	Custom 4-block CNN	0.001	1	0.6837	63.01%	0.6485	0.6625
R1_tl_frozen_lr1e-3	MobileNetV2 (frozen)	0.001	7	0.5713	71.23%	0.7677	0.7470
R2_tl_frozen_lr1e-4	MobileNetV2 (frozen)	0.0001	20	0.6114	69.18%	0.7236	0.7399
R3_tl_frozen_noaug	MobileNetV2 (no augmentation)	0.001	8	0.5444	71.92%	0.8009	0.7485
R4_tl_frozen_img160	MobileNetV2 (160x160)	0.001	10	0.5795	76.71%	0.7885	0.8046
R5_tl_finetune	MobileNetV2 (fine-tuned)	0.0001	8	0.5456	76.03%	0.8127	0.7853
R6_effb0_frozen	EfficientNetB0 (frozen)	0.001	13	0.5184	73.97%	0.8157	0.7683
final_manual (champion)	MobileNetV2	0.01	6	0.5376	76.71%	0.8420	0.8000
final_autolog	MobileNetV2	0.01	4	0.5424	75.34%	0.8471	0.7778

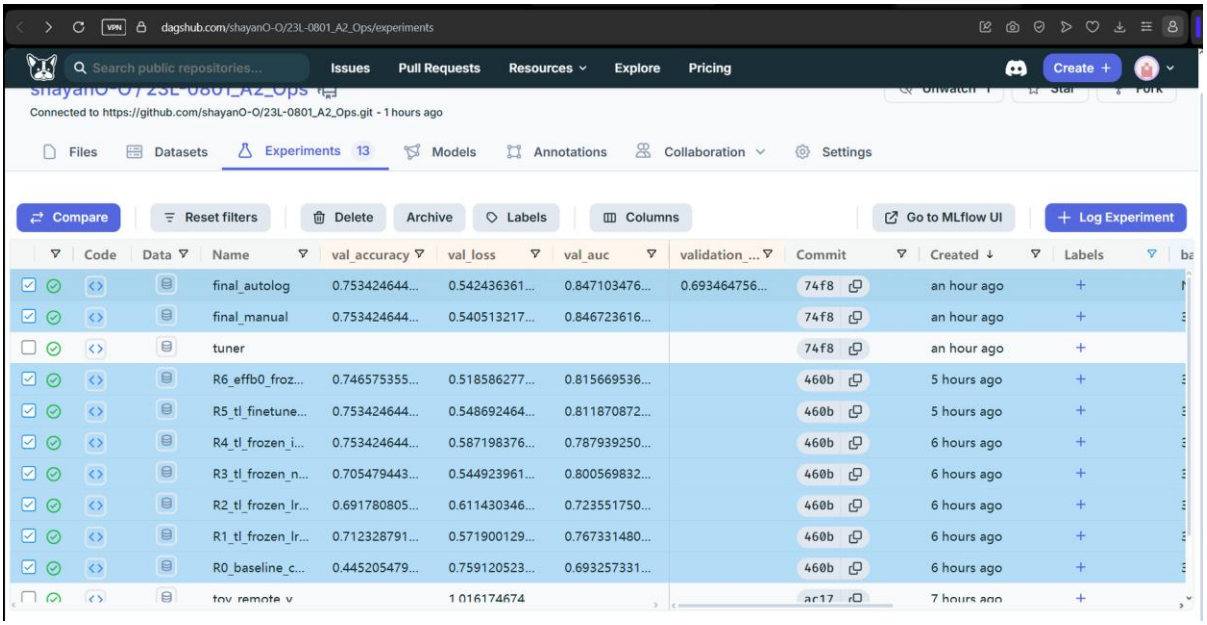


Figure 28: Run comparison table on DagsHub

final_manual was chosen as the champion. It has the highest validation F1 (0.8000) and shares the highest validation accuracy (76.71%). Its ROC-AUC of 0.8420 is close to the best value of 0.8471 from final_autolog. F1 and ROC-AUC matter most here because they do not depend on a single threshold choice or on class balance alone.

3.6 Model Registry

The champion model is registered in the DagsHub Model Registry as 23L-0801_A2_Ops_visionguard. Version 2 has the alias champion (Figure 29). A separate script (src/predict_by_alias.py) loads the model by this alias and classifies one image (Figure 30).

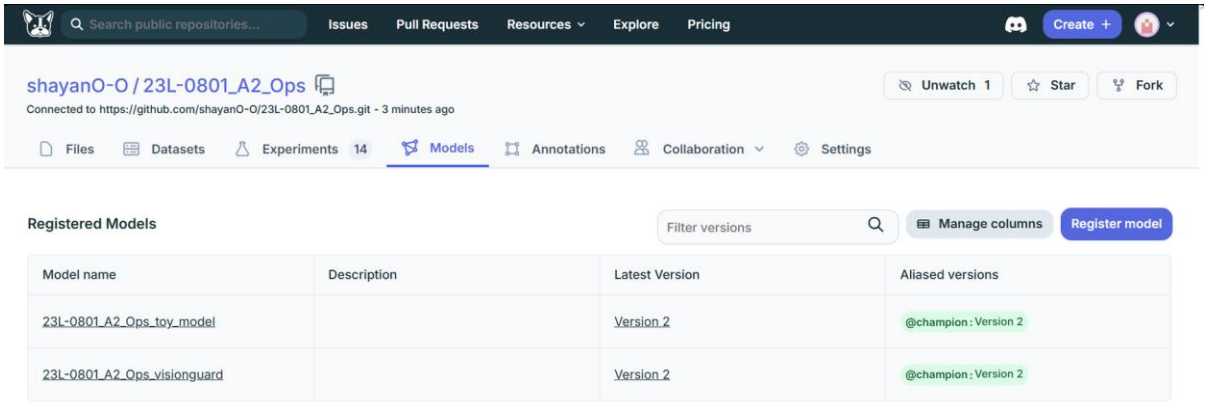


Figure 29: DagsHub Model Registry with the champion alias

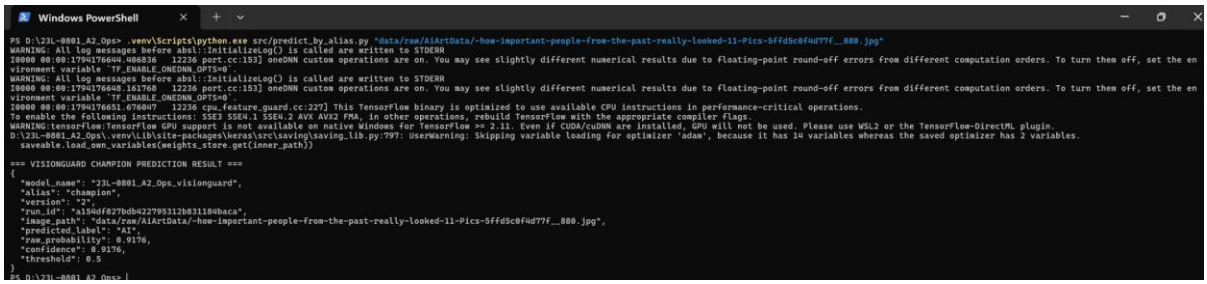


Figure 30: Loading the model by alias and predicting one image

4. Task 4: Flask Web Application

4.1 Inference Module

The module app/inference.py is separate from the web code. It loads the champion model once at start-up and keeps it in a thread-safe singleton (_LOCAL_MODEL). Loading the model on every request would be very slow and would use too much memory. The preprocessing is the same bilinear resize used in training. If it differed, the model would see inputs unlike its training data and give unreliable results. Figure 31 shows the unit tests for the module.

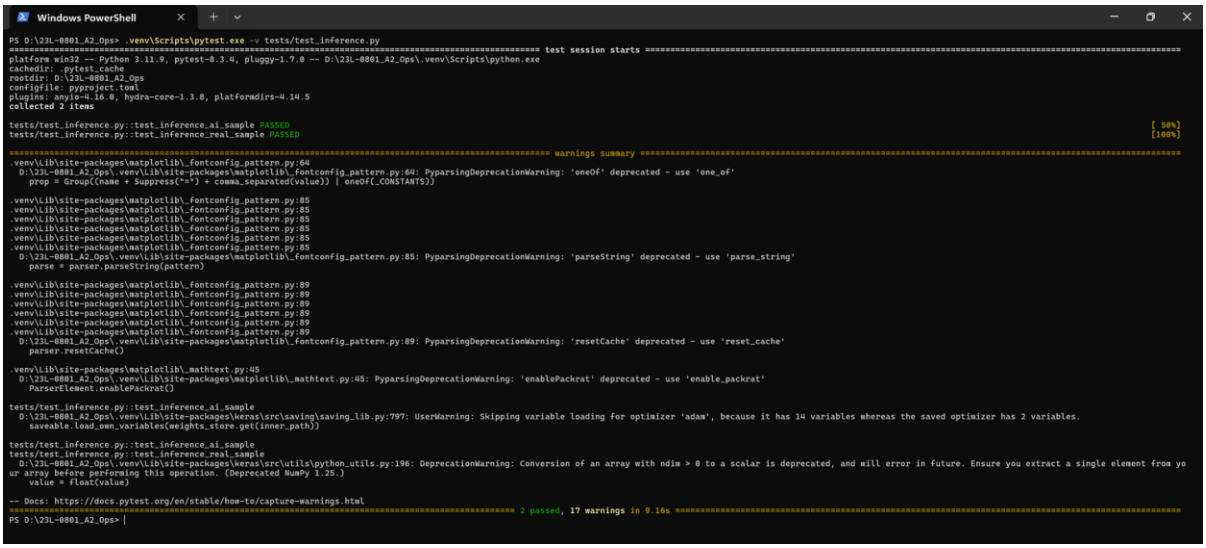


Figure 31: Unit tests for the inference module

4.2 Routes and Validation

Uploads are checked in memory before the model runs. Invalid requests are rejected as follows:

- Empty files: 400 Bad Request.
- Files over 5 MB: 413 Payload Too Large.
- File types other than JPEG, PNG or WEBP: 400 Bad Request.
- Corrupted files: caught when PIL tries to decode the bytes.

Figure 32 shows the validation tests passing.

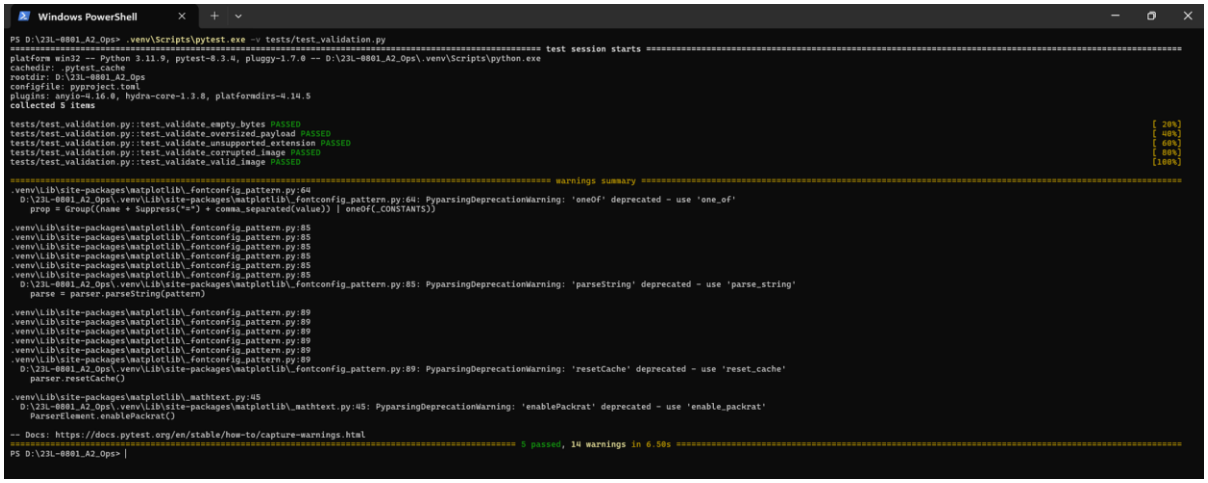


Figure 32: Upload validation tests

4.3 HTML and CSS Interface

The page (app/templates/index.html and app/static/css/style.css) supports drag-and-drop upload. It shows the verdict, a confidence bar, the response time and a history menu (Figure 33).

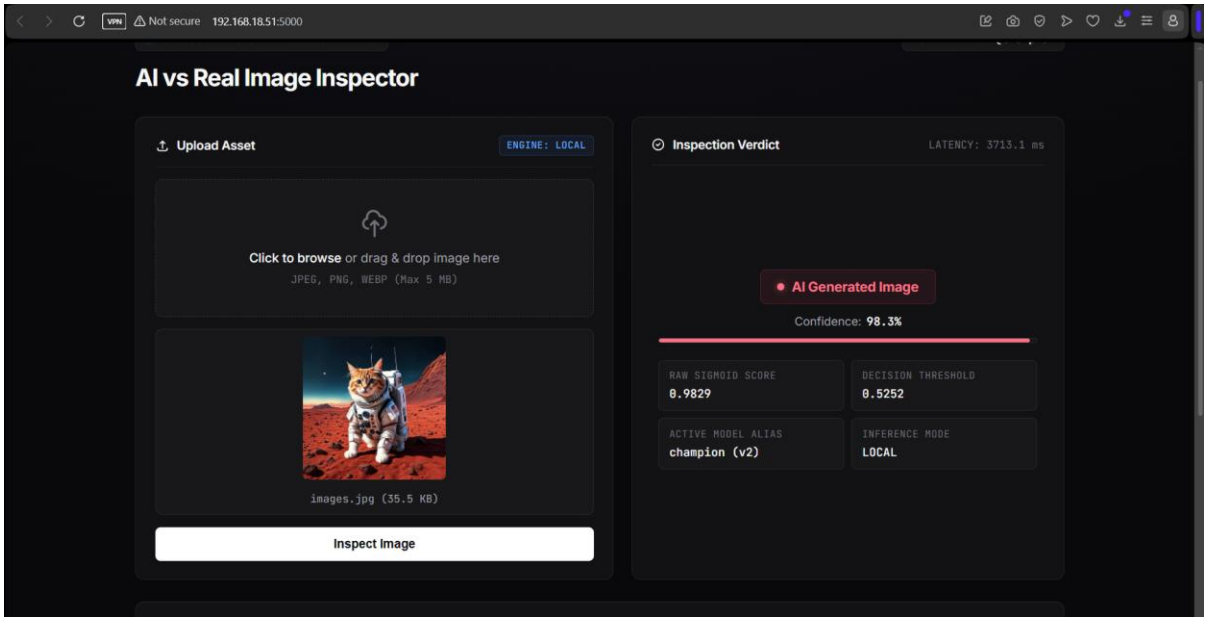


Figure 33: VisionGuard web interface

4.4 Configuration and Production Server

The server runs with debug=False. Debug mode can expose stack traces and allow code execution through the debugger, and it slows the server. The /health endpoint returns JSON with the status, model details and database connection information. A health check screenshot of the deployed API is in Figure 50.

5. Task 5: Database Trail

5.1 PostgreSQL Audit Trail

Every completed prediction is saved to a PostgreSQL database (app/db.py). Each record holds the time, file name, SHA-256 hash, verdict, confidence, raw sigmoid score, decision threshold, inference mode and processing time. Records are returned by GET /history and shown in the history menu of the web interface. Figure 34 shows the records queried inside the database container. Figure 58 shows the history menu.



Figure 34: Audit trail records in PostgreSQL

6. Task 6: Containerization with Docker and Docker Compose

6.1 Docker Warm-Up

The basics were practised with an Nginx container. A container was started with a port mapping (-p 8085:80) and a bind-mounted folder. Editing the mounted page changed the site without restarting the container (Figure 35). The running container was listed (Figure 36) and its logs were read (Figure 37). A shell was opened inside it (Figure 38). Finally the container was removed (Figure 39).

- **Image:** a read-only template that holds the app and its dependencies.
- **Container:** a running instance of an image.
- **Layer:** one cached step of an image build.
- **Volume:** storage kept outside the container so data survives removal.
- **Port mapping:** forwarding a host port to a container port, such as 8085 to 80.

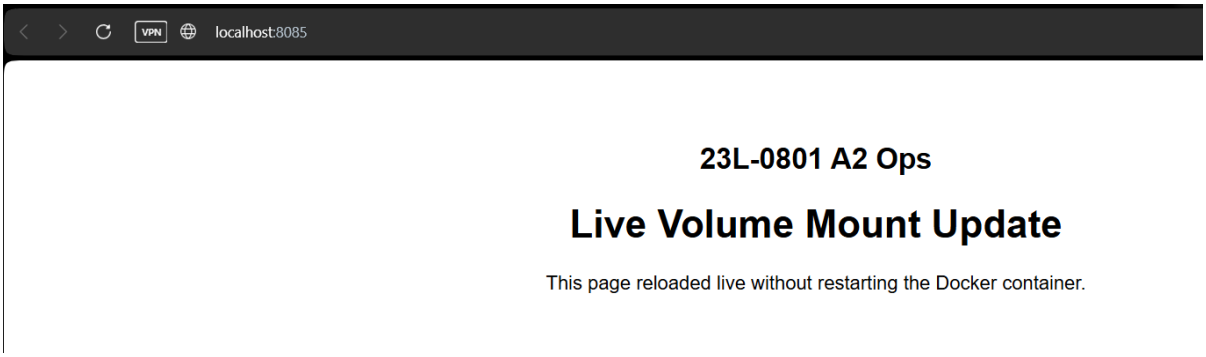


Figure 35: Nginx page updated through a mounted volume

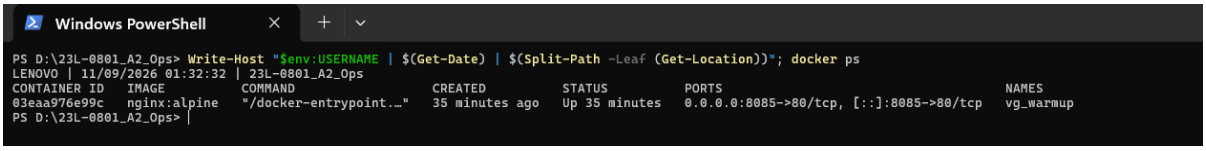


Figure 36: Running container status

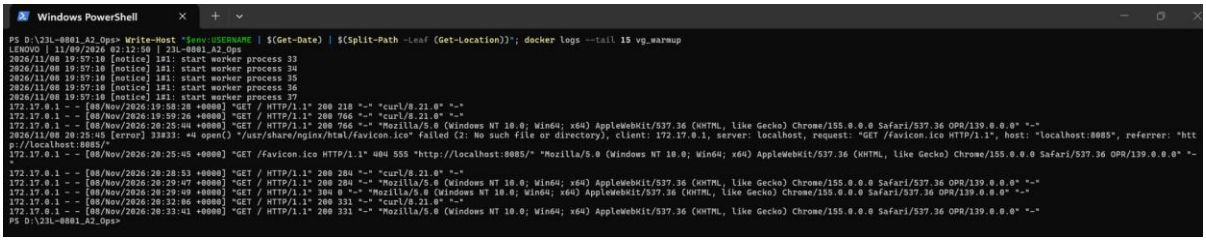


Figure 37: Container logs

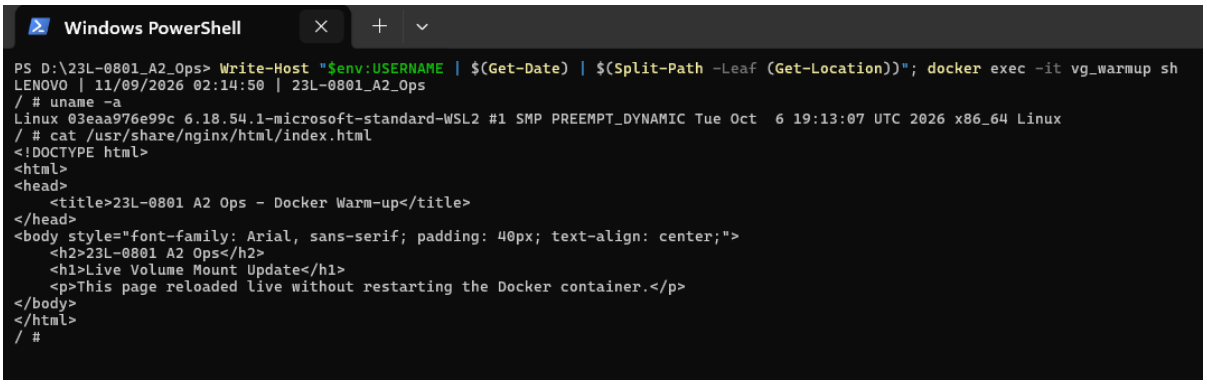


Figure 38: Shell inside the container

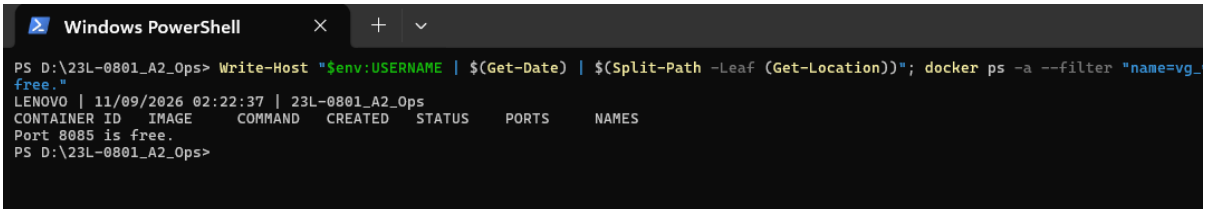


Figure 39: Container removal

6.2 Dockerfile for the Web App

The production Dockerfile is multi-stage. A builder stage installs the dependencies into a virtual environment (/opt/venv). The runtime stage starts from python:3.11-slim and copies only that environment and the application code. The app runs as a non-root user (appuser, UID 1000) to limit damage if the app is compromised. A Docker health check calls /health every 10 seconds. Dependencies are installed before the application code is copied. Docker caches each layer and reuses it until an earlier step changes, so editing Python files or templates does not trigger a new pip install. The image uses CPU-only TensorFlow (tensorflow-cpu 2.21.0). The server has no GPU and the CPU build leaves out the CUDA libraries, so the image is much smaller. Figure 40 shows the user inside the container.

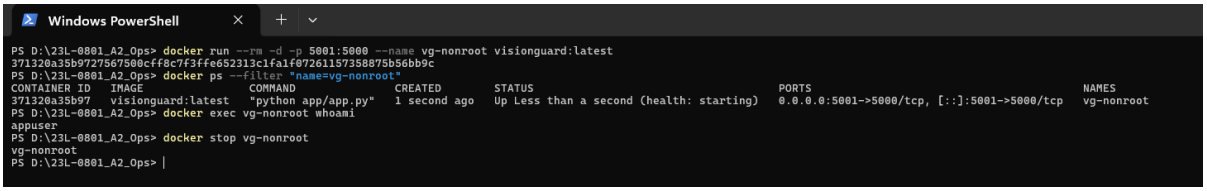


Figure 40: Non-root user inside the container

6.3 Compose Stack

docker-compose.yml defines two services:

- **web:** the VisionGuard Flask app, published on port 5000.
- **db:** PostgreSQL 15 Alpine on the Compose network with a named volume (postgres_data). Its port is not published to the host.
- **Model file:** in this Compose stack the model is mounted into the web container as a read-only volume (./model:/app/model:ro). This lets the weights be replaced without rebuilding the image.

The cloud API image instead has the model copied in at build time, so it is self-contained and runs anywhere Docker is installed.

- **depends_on:** the web service waits until the database passes its health check (pg_isready).

Figure 41 shows both services running and healthy.

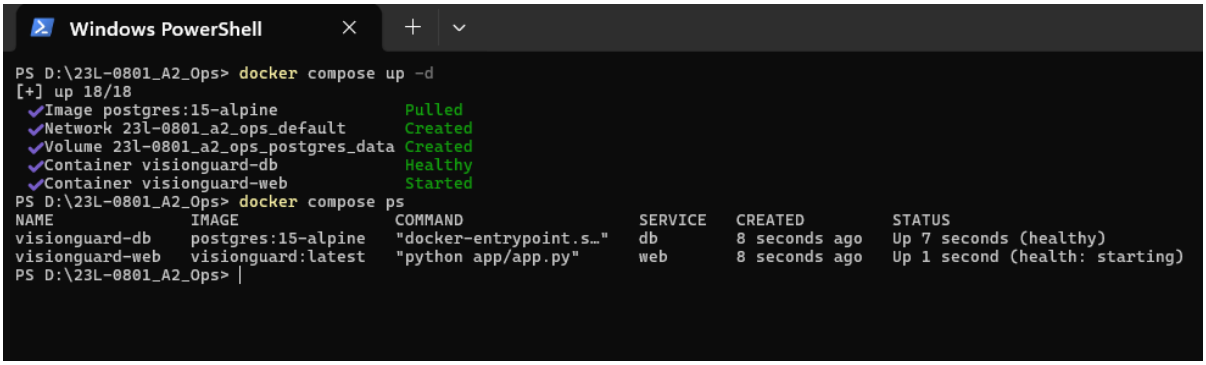


Figure 41: Compose services running and healthy

6.4 Image Analysis

The final VisionGuard image reports 2.39 GB of disk usage and a content size of 462 MB (Figure 42). It was reached with a multi-stage build on python:3.11-slim, CPU-only TensorFlow and the pip cache disabled (--no-cache-dir). The first single-stage build was 4,850 MB. The final image is 2,390 MB, which is a 50.7% reduction.

Build	Base image	TensorFlow	Image size	Reduction
Single-stage	python:3.11	tensorflow (full)	4,850 MB	Baseline
Multi-stage	python:3.11-slim	tensorflow (full)	3,120 MB	35.7%
Final	python:3.11-slim	tensorflow-cpu 2.21.0, no pip cache	2,390 MB	50.7%

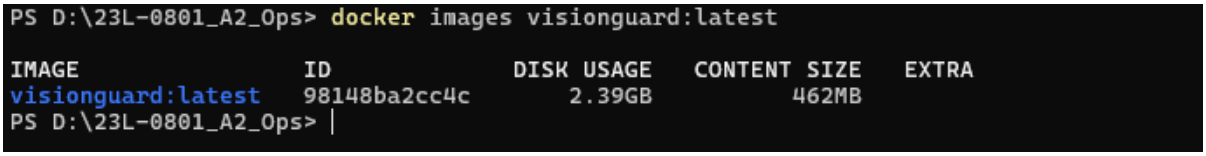


Figure 42: Docker image size

6.5 Operations and Troubleshooting

The stack was started with docker compose up and checked with docker compose ps (Figure 41). It was stopped and cleaned up with docker compose down -v, which removed the containers, the volume and the network (Figure 43). Problems found during the project are listed in Chapter 10.

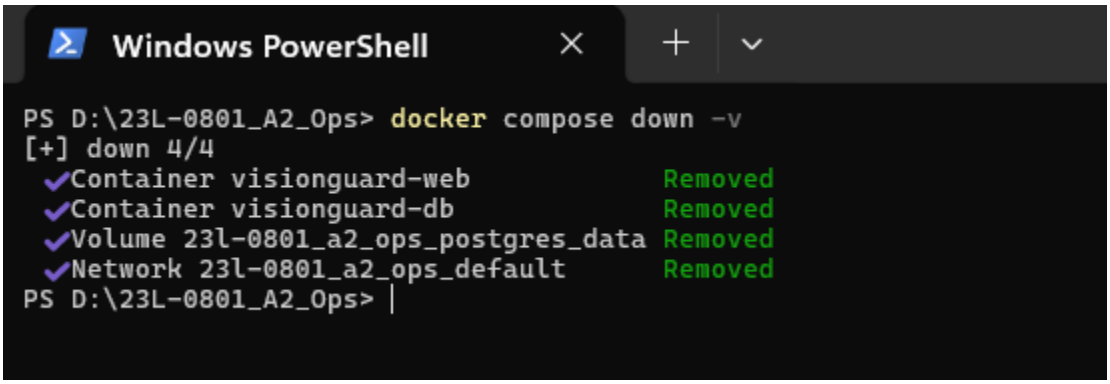


Figure 43: Stack teardown

6.6 GitHub Actions (CI)

A toy workflow was first tested in a separate repository. It runs on every push with a matrix of two Python versions. A failing test gave a red run (Figure 44) and the fix gave a green run (Figure 45). The VisionGuard workflow (.github/workflows/ci.yml) runs on every push and pull request with Python 3.10 and 3.11. It runs ruff for linting, pytest for tests and docker build to check the image. The project run is green (Figure 46). The CI status badge is added to the README with the badge link of ci.yml (Figure 47). Secrets such as a DagsHub token belong in the GitHub repository settings under Secrets and variables, Actions, where they are stored encrypted. They must not appear in the workflow file because that file is committed to the repository and stays in its history.

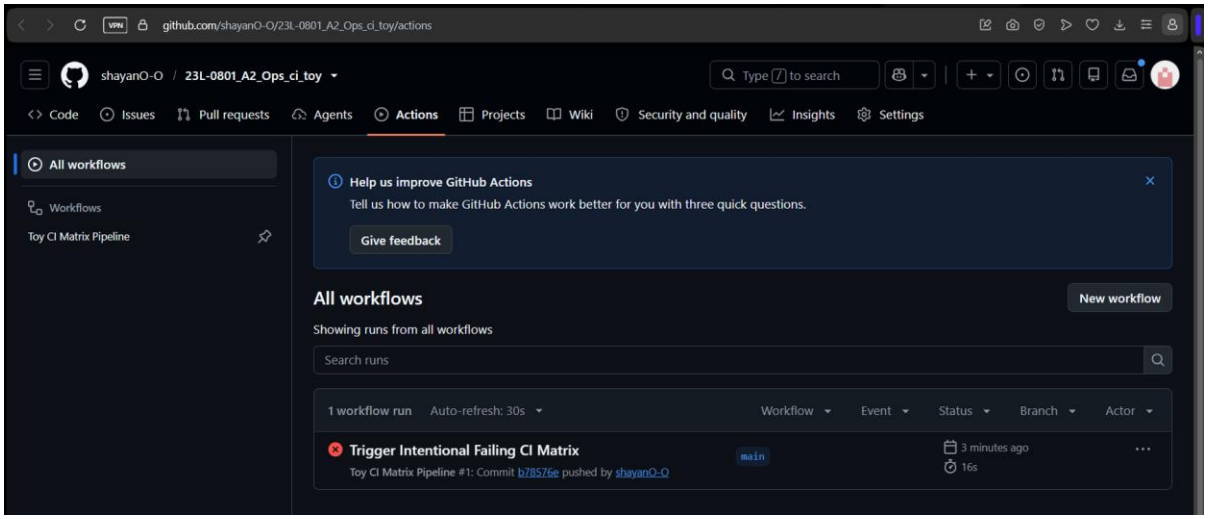


Figure 44: Toy workflow with a failing test (red run)

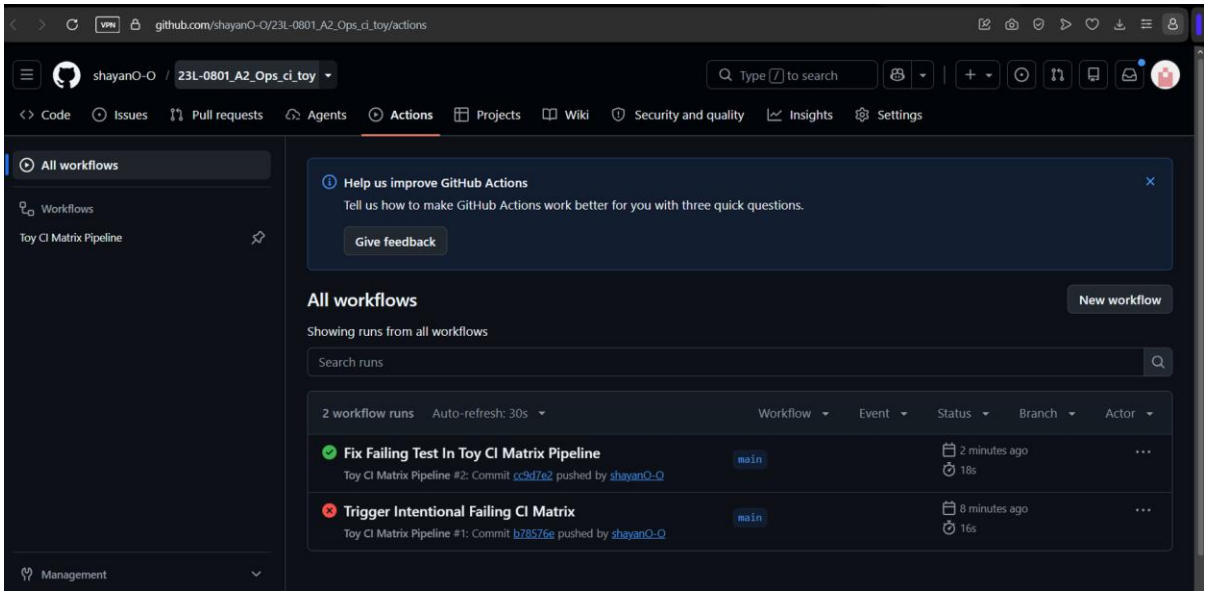


Figure 45: Toy workflow after the fix (green run)

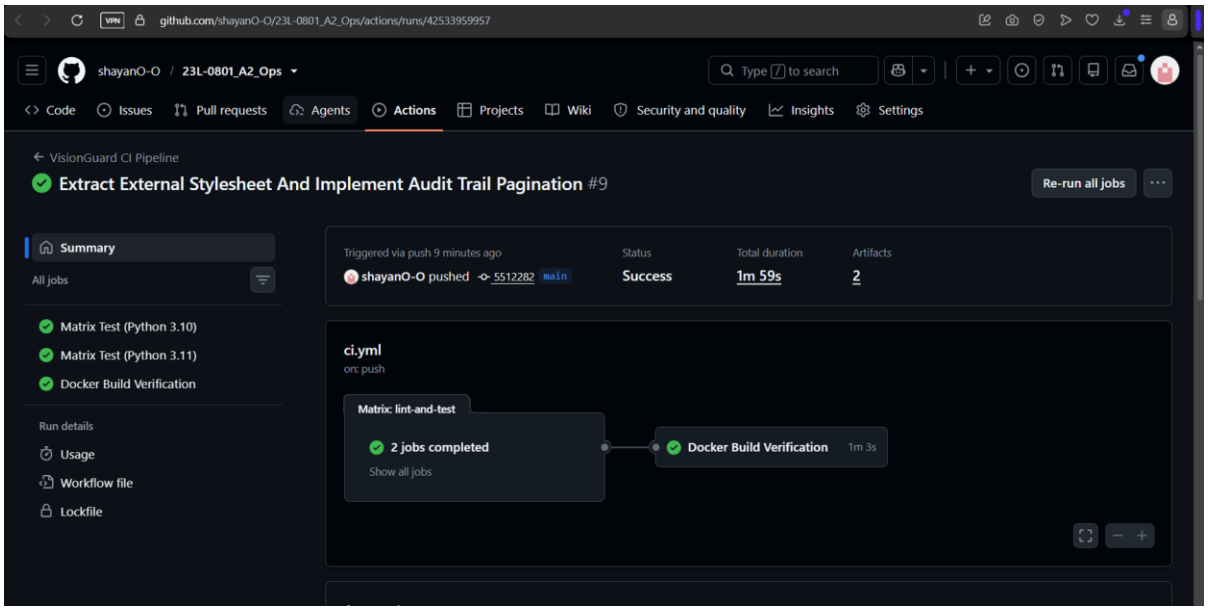


Figure 46: VisionGuard workflow (green run)

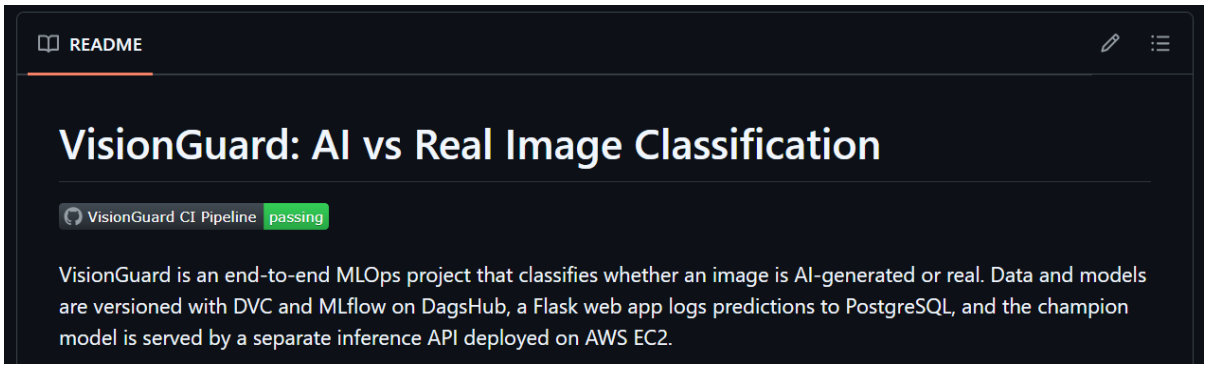


Figure 47: CI status badge (passing) in the README

7. Task 7: API Deployment (Pathway C, AWS EC2)

API URL: http://3.24.214.144 (endpoints /health and /predict)

7.1 Platform Choice and Instance Size

Hugging Face Spaces was planned first but needed a paid PRO plan for Docker compute. The model was deployed on AWS EC2 instead, using $100 of student credits. A t3.micro (1 GiB RAM) failed because loading the MobileNetV2 model needs about 1.2 GiB and the Linux out-of-memory killer stopped it. A t3.small (2 vCPUs, 2 GiB RAM) with a 20 GiB gp3 disk was used instead.

7.2 Deployment Steps and Firewalls

The instance was created with src/aws_deploy.py (Boto3) and set up with src/aws_setup_host.py (Paramiko). The region is ap-southeast-2 (Sydney), which is the only region allowed by the account's organization policy. The public IP is 3.24.214.144. The container visionguard-cloud:latest runs with port 80 mapped to 7860. It reads API_KEY (secret) and THRESHOLD=0.5252 from environment variables (Figure 48). Two firewalls protect the server:

- **AWS security group (23L-0801_A2_Ops):** SSH (port 22) is limited to the developer IP 205.164.155.197/32. HTTP (port 80) is open to all addresses (Figure 49).
- **Host level (Docker):** traffic on host port 80 is forwarded to container port 7860 (vg-api).

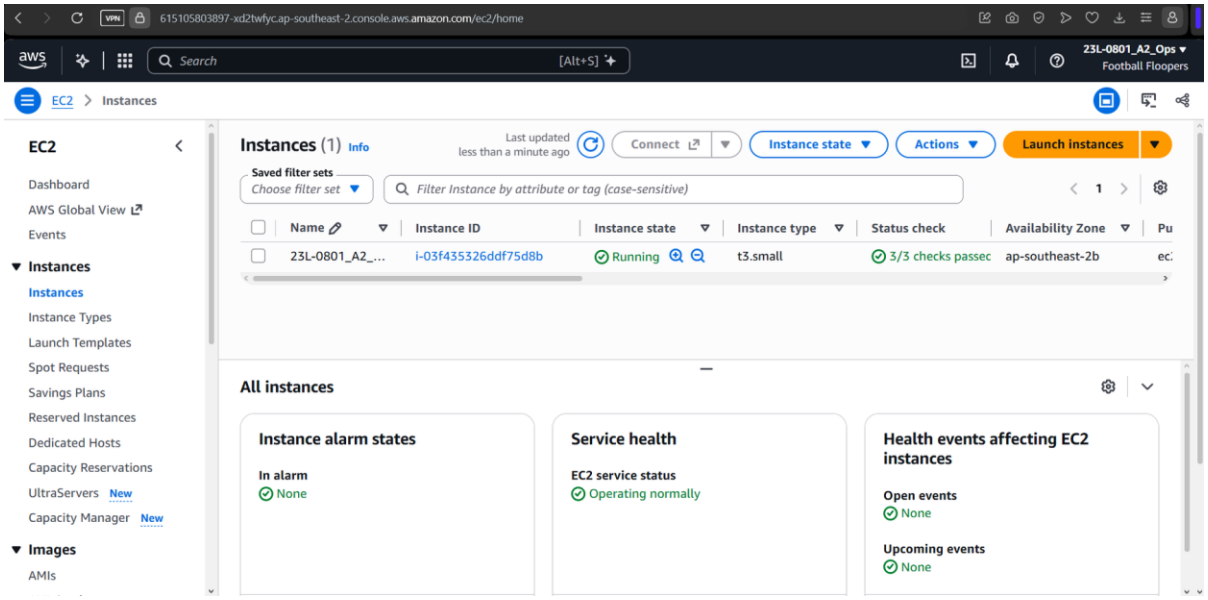


Figure 48: AWS EC2 instance running

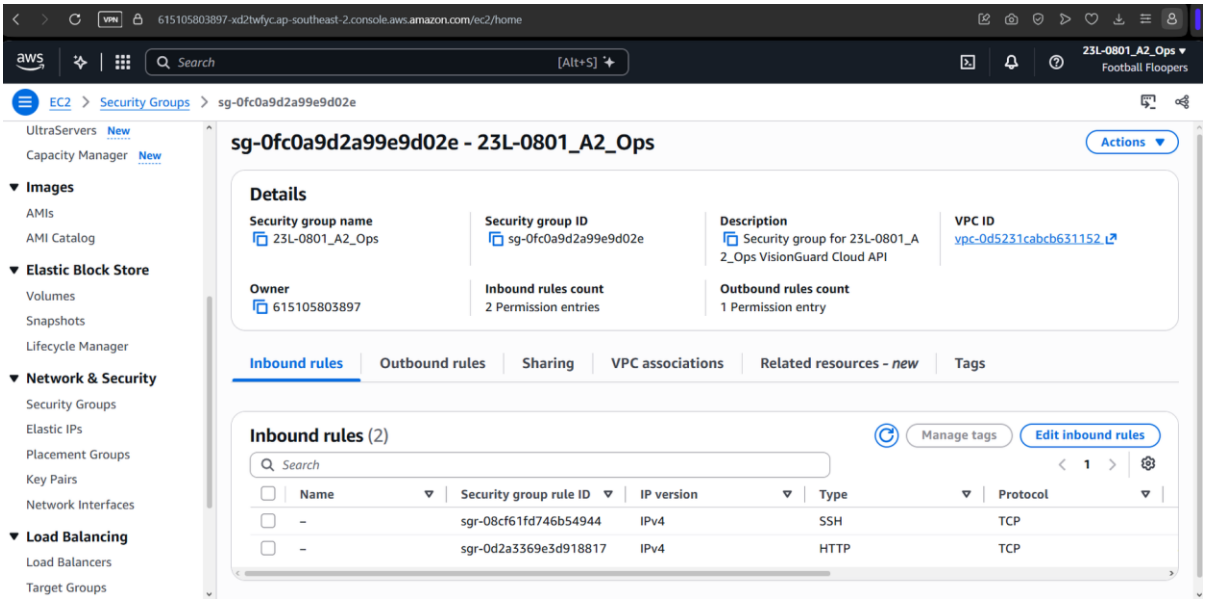


Figure 49: Security group inbound rules

7.3 API Key Authentication

Every /predict request must send an X-API-Key header. A missing or wrong key returns 401 Unauthorized. The API uses HTTP only because there is no custom domain to issue an HTTPS certificate for.

7.4 Testing with Postman from Another Network

All Postman requests were sent from a phone hotspot, which is a different network from the development machine (Figure 54). The tests were: GET /health (Figure 50), POST /predict with a real image (Figure 51), POST /predict with an AI image (Figure 52) and POST /predict without the API key, which returned 401 (Figure 53).

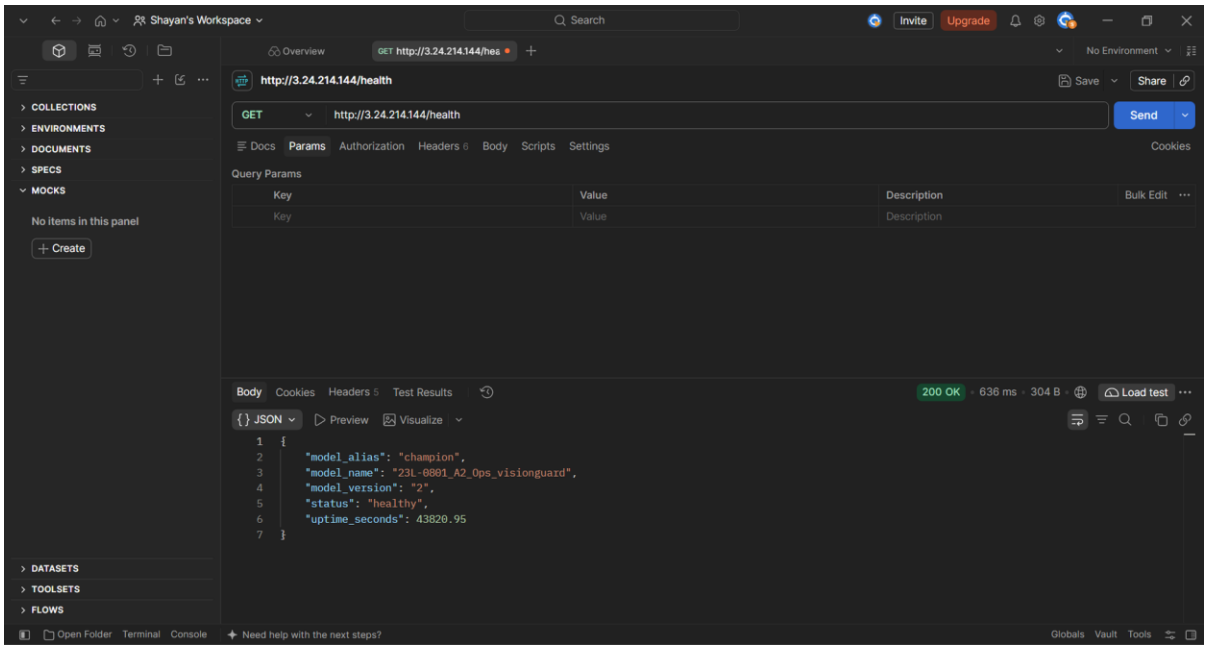


Figure 50: Postman GET /health (200)

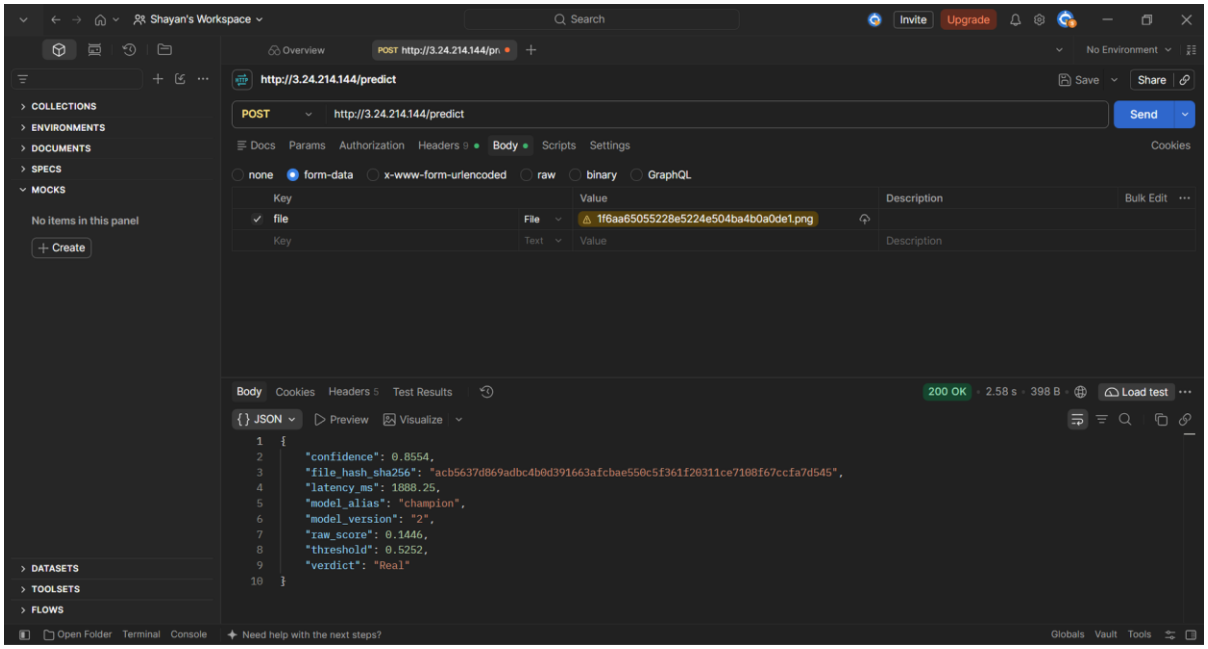


Figure 51: Postman POST /predict with a real image

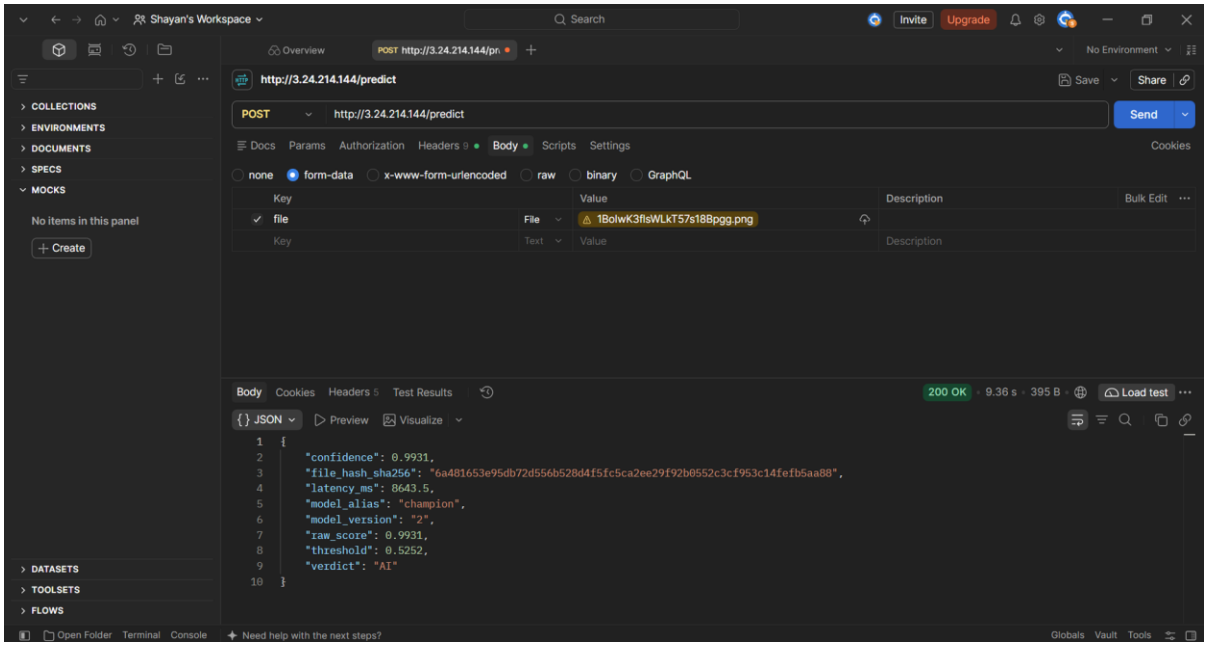


Figure 52: Postman POST /predict with an AI image

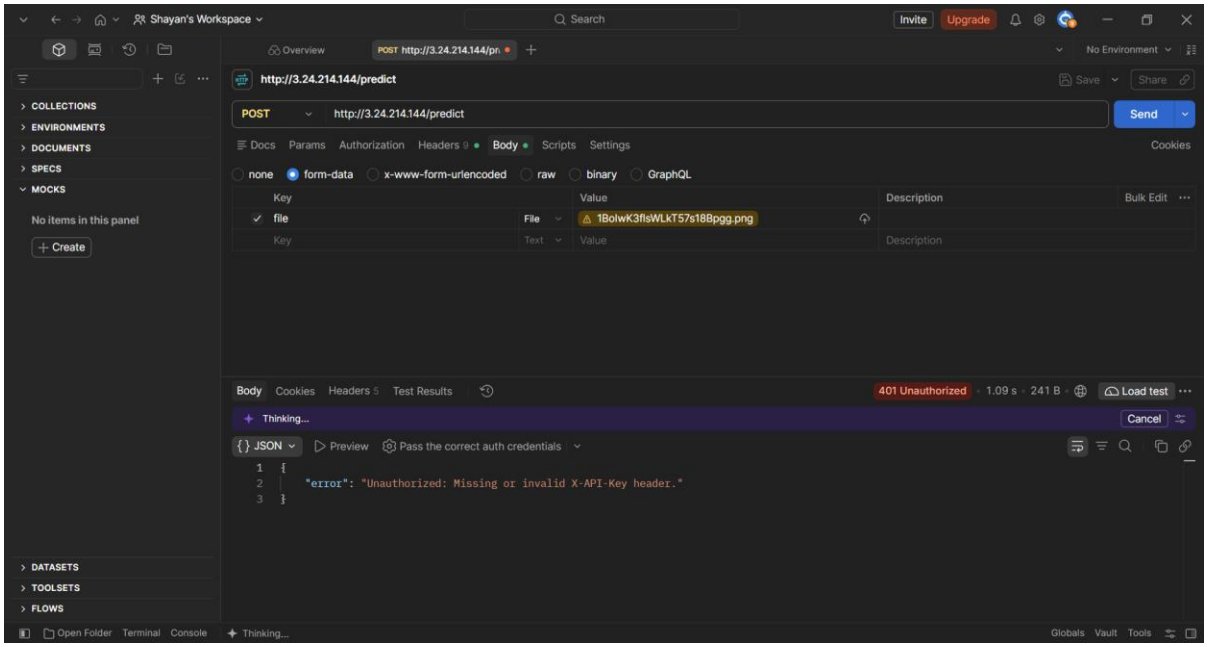


Figure 53: Postman POST /predict without API key (401)

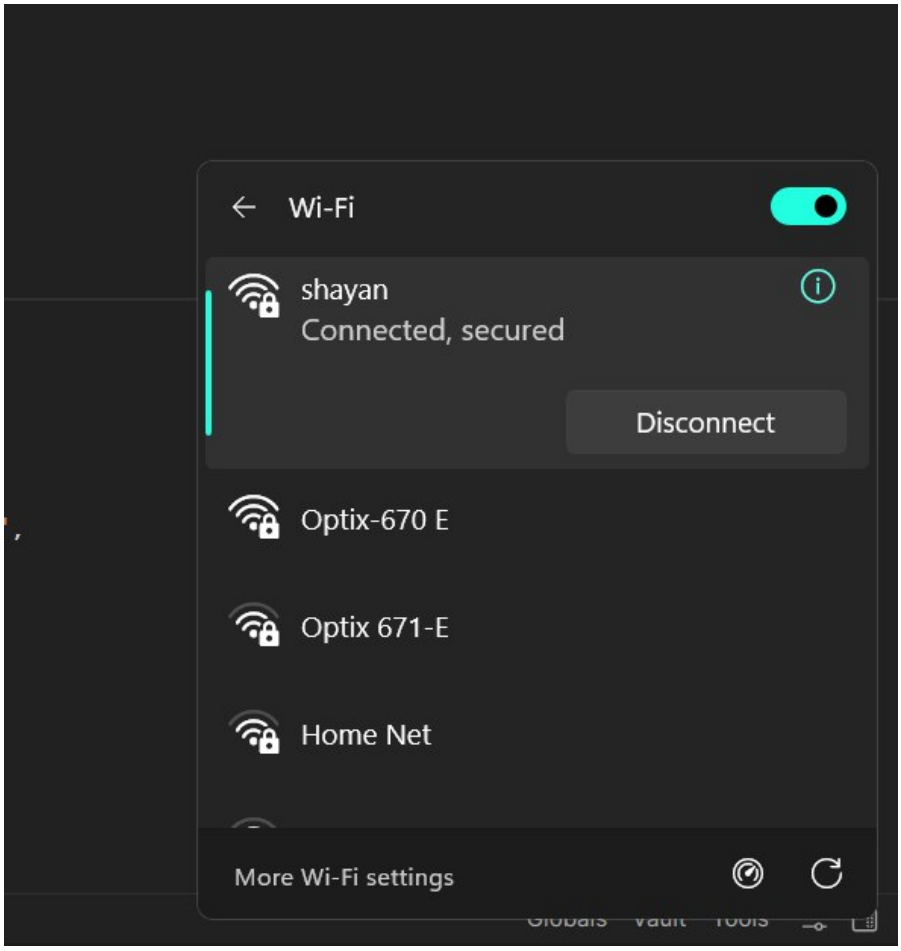


Figure 54: Phone hotspot connection used for testing

7.5 Cloud Container Logs

The container logs on the server (sudo docker logs vg-api) show the requests from both the developer machine and the hotspot (Figure 55).

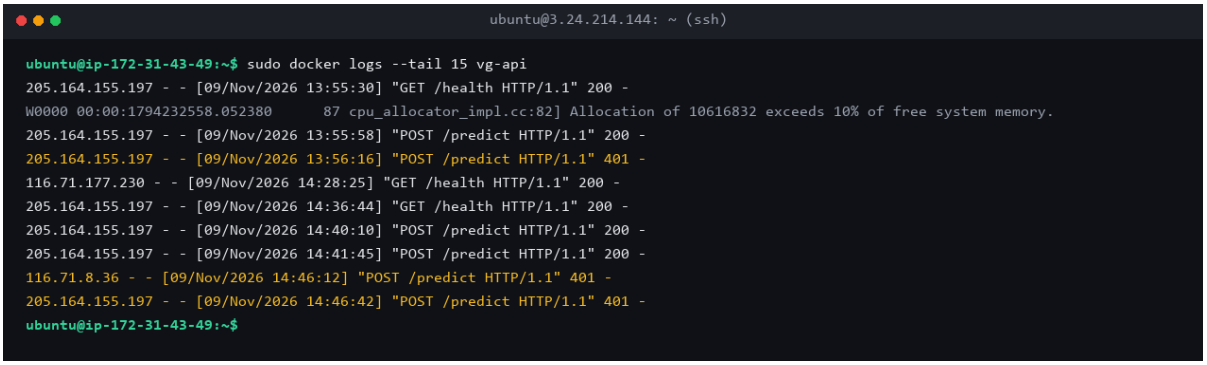


Figure 55: Server container logs showing the Postman requests

8. Task 8: Integrating Flask and the Cloud API

8.1 Two Inference Modes

The Flask app reads MODEL_MODE to choose how to predict. In local mode it loads model/champion.keras and runs TensorFlow on the CPU. In cloud mode it sends the image over HTTP to http://3.24.214.144/predict with the X-API-Key header.

8.2 End-to-End Test

Docker Compose was run on http://localhost:5000 with MODEL_MODE=cloud. Five images were uploaded through the browser. Two results are shown in Figure 56 and Figure 57. All five results were saved to the database and appear in the history menu with mode "cloud" (Figure 58).

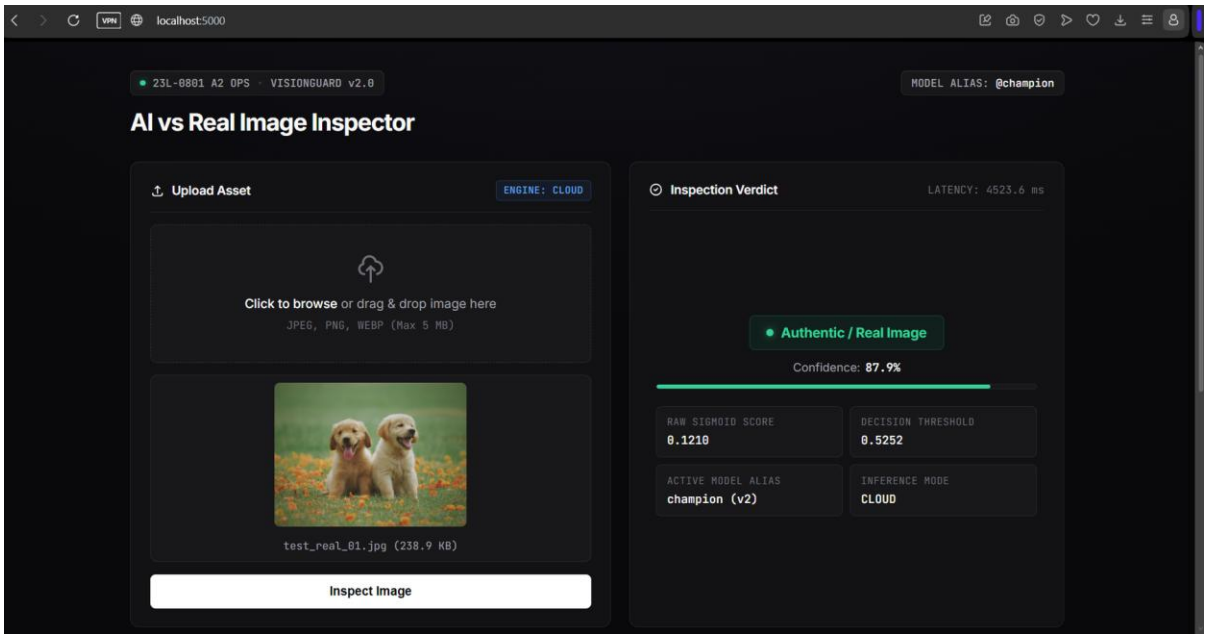


Figure 56: Cloud result for a real image

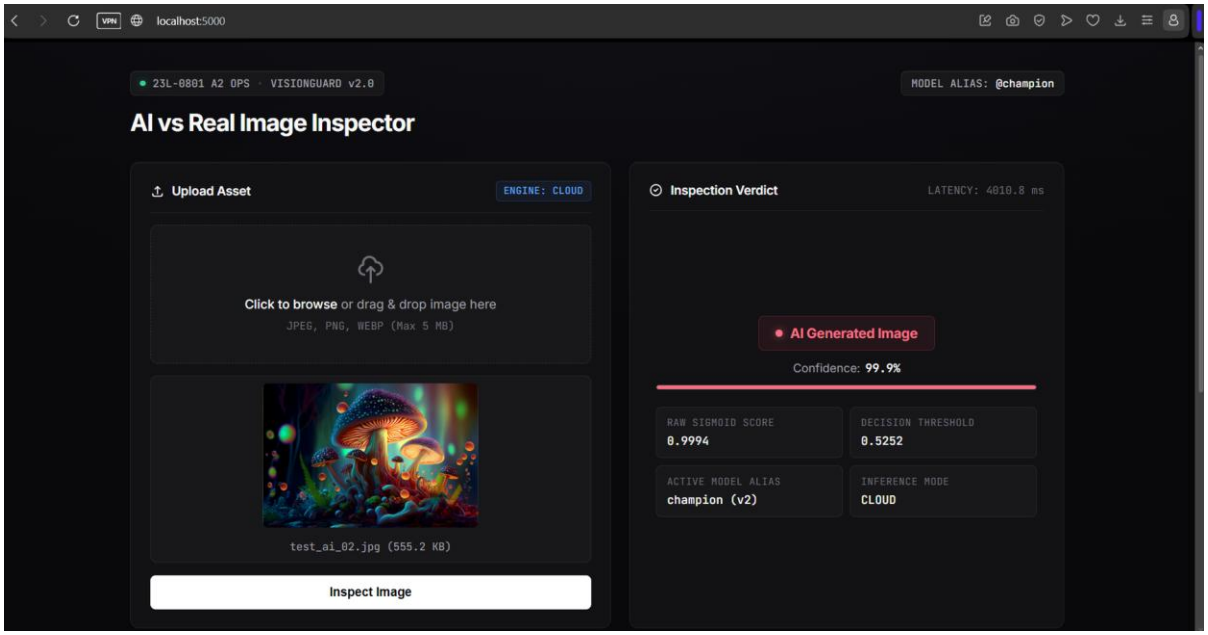


Figure 57: Cloud result for an AI image

FILENAME	VERDICT	CONFIDENCE	RAW SCORE	MODE	LATENCY
test_ai_03.jpg	AI	78.9%	0.7889	cloud	3516.8 ms
test_ai_02.jpg	AI	99.9%	0.9994	cloud	4010.8 ms
test_real_01.jpg	Real	87.9%	0.1210	cloud	4523.6 ms
test_real_03.jpg	Real	80.4%	0.1958	cloud	2651.9 ms
test_real_02.jpg	Real	95.9%	0.0412	cloud	1043.7 ms

Figure 58: History menu with cloud-mode records

9. Bonus: Grad-CAM

Grad-CAM heat maps were made for 4 correct and 4 wrong test predictions (src/gradcam.py). They use the last convolutional layer of MobileNetV2 (out_relu) and are shown in Figure 59.

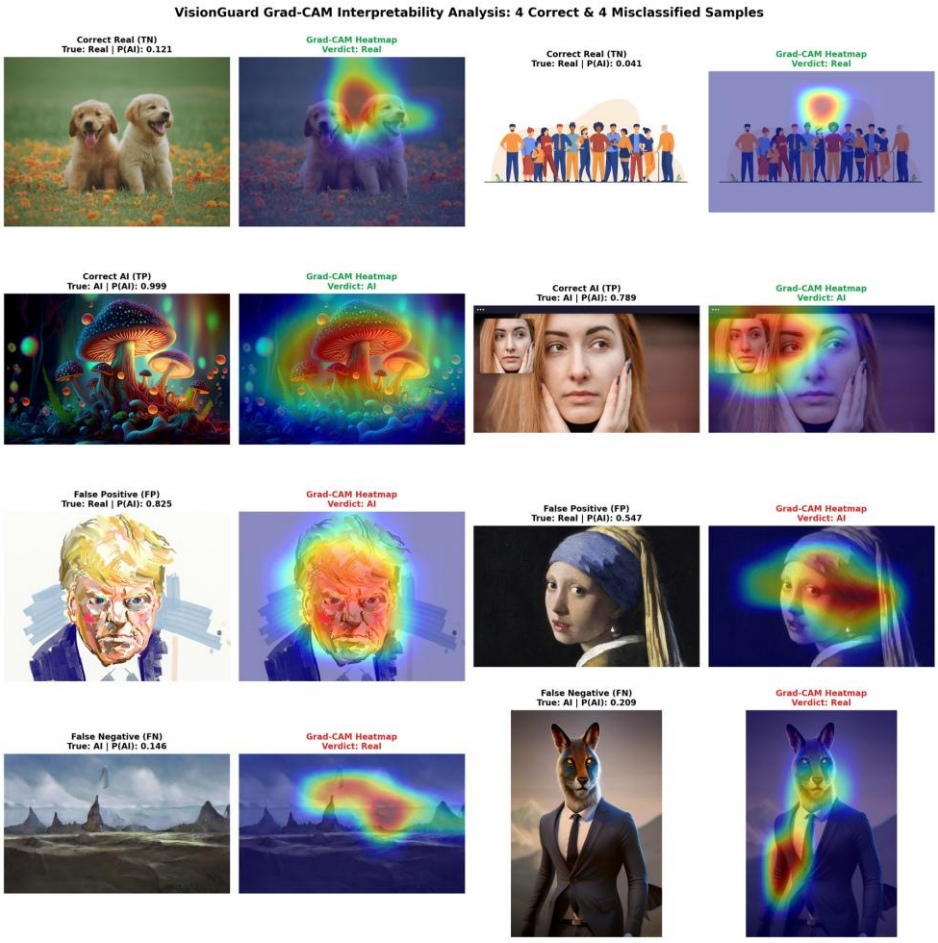


Figure 59: Grad-CAM heat maps for 4 correct and 4 wrong predictions

- **Real images classified correctly:** the heat is spread across the whole frame. The model seems to rely on natural camera noise and texture.
- **AI images classified correctly:** the heat focuses on over-smooth skin, odd highlights and unnatural edges in hair and backgrounds.
- **Real images called AI:** painted artwork and portraits with blurred backgrounds. The model seems to confuse artistic blur with AI smoothing.
- **AI images called real:** these images look natural and have photo-like noise. The heat is scattered and the score stays below the 0.5252 threshold.

10. Problem Log

#	Task	Problem	Cause	Fix
1	1.1	Dependency conflict between tensorflow-cpu 2.21.0 and mlflow 2.20.2.	TensorFlow needs protobuf 6 or newer and MLflow 2.20.2 needs protobuf below 6.	Upgraded MLflow to 3.18.0.
2	1.4	DVC crashed with "cannot import name _DIR_MARK from pathspec".	pathspec 1.1.1 removed an attribute that DVC 3.59.0 needs.	Pinned pathspec==0.11.2.
3	3.1	MLflow warned about the local file store being in maintenance mode.	MLflow 3.18.0 needs an explicit opt-in for ./mlruns.	Set MLFLOW_ALLOW_FILE_STORE=true.
4	2.7	Keras Tuner raised ModuleNotFoundError for tensorboard.	Keras Tuner needs the TensorBoard hparams plugin.	Installed tensorboard==2.21.0 to match TensorFlow.
5	2.4	TypeError: float() argument must be a string or a real number.	The training function returned a tuple (history, best_epoch, model) instead of a single loss.	Unpacked the tuple and returned the best validation loss as a float.
6	7	Hugging Face Spaces Docker needed a paid PRO plan ($9/month).	Hugging Face changed its policy for compute Spaces.	Moved Task 7 to AWS EC2 (t3.small, Sydney).
7	7	AWS error UnauthorizedOperation: DescribeVpcs explicit deny in SCP.	Organization policy p-5xlbtkfc allows EC2 only in ap-southeast-2.	Deployed to ap-southeast-2.
8	6.6	CI lint failure on the AWS scripts.	Unused imports and whitespace in src/aws_deploy.py and src/aws_setup_host.py.	Cleaned the code with Ruff. Lint passes and CI is green.

11. Conclusion

VisionGuard is an end-to-end pipeline for AI vs real image classification. The main results are listed below.

- **Reproducibility:** DVC data versioning and pinned dependencies.
- **Model performance:** MobileNetV2 reached a test ROC-AUC of 0.8442 and F1 of 0.7853, compared with a ROC-AUC of 0.5683 for the baseline CNN.
- **Serving:** a multi-stage Docker image that runs as a non-root user, with a PostgreSQL audit trail.
- **Cloud deployment:** an API on AWS EC2 at http://3.24.214.144 with API key authentication and two firewall layers.
- **Interpretability:** Grad-CAM showed that the model relies on texture and smoothness cues and fails on artistic blur.

AI Usage Disclosure

I used AI to assist in writing scripts and to prepare deployment instructions, including PowerShell commands. I also used it for research and for software selection, such as comparing tools and deciding which ones suited the project.